

מערכים

0	1	2	3	4
3	23	89	644	76

0	1	2	3
"דנה"	"דני"	"חיים"	"משה"

0	1	2	3	4	5	6	7
8	-7.9	2.33	-4.3	77.3	87.0	6.77	4.5

ביחידה זו נלמד:

- **דוגמאות אלגוריתמיות:**

- ✓ סיבוב מערך
- ✓ הדפסת תווים
- ✓ מיזוג מערכים ממויינים
- ✓ מספר ארוך מאד
- ✓ שאלת בגרות

- **מיונים וחיפושים**

- ✓ סוגי מיונים
- ✓ מיון בועות
- ✓ מיון בחירה
- ✓ חיפוש בינארי

- מהו מערך
- כיצד מערך נראה בזיכרון
- גישה לאיברי המערך
- אתחול מערך
- חריגה מגבולות המערך
- השמת מערכים
- מערכים כפרמטרים לפעולות
- החזרת מערך מפעולה
- מערך מונים
- מערך צוברים

```
static void CountAboveAverage()
```

```
{
```

```
    int num1, num2, /*...*/num20, sum = 0;
```

```
    double average;
```

```
    int countAboveAverage = 0;
```

```
    Console.WriteLine("Please insert 20 numbers: ");
```

```
    num1 = int.Parse(Console.ReadLine());
```

```
    num2 = int.Parse(Console.ReadLine());
```

```
    /*...*/
```

```
    num20 = int.Parse(Console.ReadLine());
```

```
    sum = num1 + num2 /*+ ...*/ + num20;
```

```
    average = sum / 20.0;
```

```
    if (num1 > average)
```

```
        countAboveAverage++;
```

```
    if (num2 > average)
```

```
        countAboveAverage++;
```

```
    /*...*/
```

```
    if (num20 > average)
```

```
        countAboveAverage++;
```

```
    Console.WriteLine(countAboveAverage + " values are above the average " + average);
```

```
}
```

מוטיבציה

- תוכנית הקוראת 20 מספרים ומציגה כמה מספרים גדולים מהממוצע ?

התוכנית מסורבלת ומייגע לכתוב אותה, בייחוד כי יתכן גם שנרצה ממוצע של 100 מספרים, או אפילו יותר...

הגדרה

- מערך הוא אוסף של משתנים מאותו הסוג עם שם אחד, ובעלי תפקיד זהה.

- דוגמאות :

`int[] numbers = new int[20];` ← מערך של 20 מספרים

`char[] letters = new char[10];` ← מערך של 10 תווים

`string[] words = new string[15];` ← מערך של 15 מילים

`<type>[] <var-name> = new <type>[<size>];` ← ובאופן כללי:

- גודל המערך בזיכרון : **`<SIZE>*גודל הטיפוס`**

הגדרת מערך בזיכרון

- יצירת מערך מורכבת מ-3 שלבים:

```
int[] arr = new int[3];
```

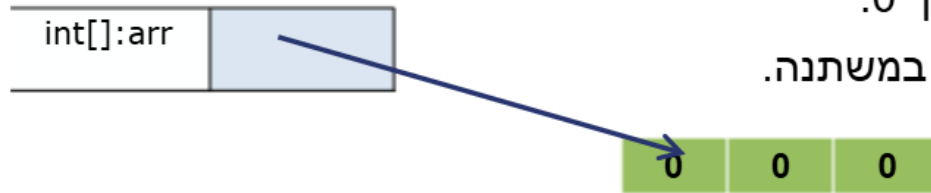
1. יצירת משתנה שיכיל את ההפניה למערך עצמו.

2. הקצאת שטח המערך, בהתאם לגודל המבוקש.

ניתן גם לאתחל את המערך, ישירות בזמן ההקצאה.

ברירת המחדל עבור איתחול האיברים, הערך 0.

3. השמה של ההפניה לעצם (לשטח שהוקצה) במשתנה.



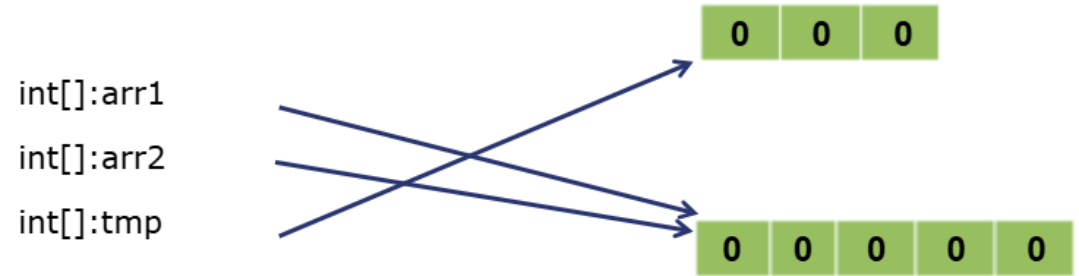
מערך הוא עצם.

הוא מורכב מ-2 חלקים:

- **משתנה** המכיל את ההפניה לעצם (למערך)
- **השטח** שהוקצה לעצם בזכרון.

מה עושה הקוד הבא ?

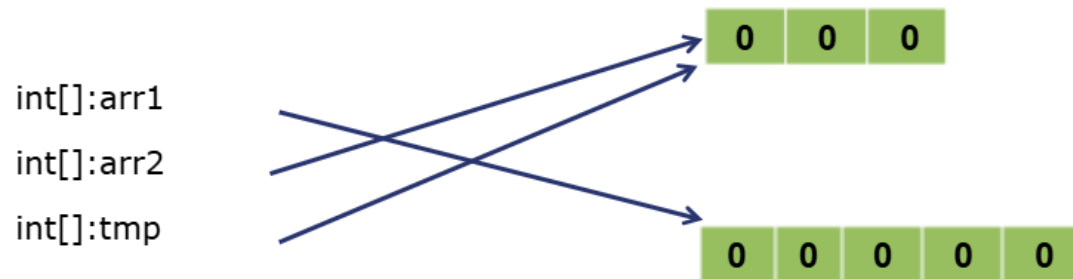
```
static void ArraysInMemory()  
{  
    int[] arr1 = new int[3];  
    int[] arr2 = new int[5];  
    int[] tmp = arr1;  
    arr1 = arr2;  
}
```



ההוראה `new` פרושה הקצאת שטח.
ללא `new` – מדובר בק, בהפניה נוספת לאותו העצם.

מה עושה הקוד הבא ?

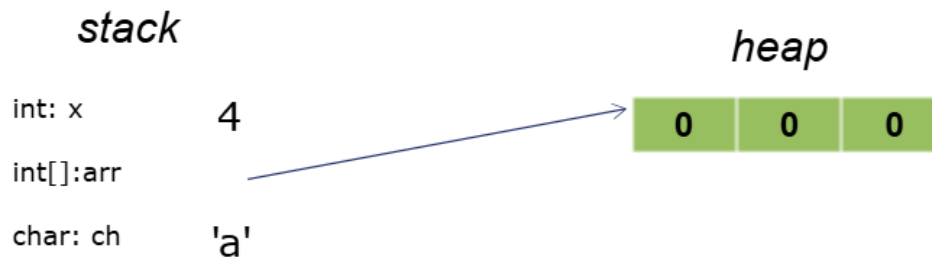
```
static void ArraysInMemory()  
{  
    int[] arr1 = new int[3];  
    int[] arr2 = new int[5];  
    int[] tmp = arr1;  
    arr1 = arr2;  
    arr2 = tmp;  
}
```



ההוראה `new` פרושה הקצאת שטח.
ללא `new` – מדובר בק בהפניה נוספת לאותו העצם.

שמירת מערך בזיכרון

```
static void ArrayInMemory()  
{  
    int x = 4;  
    int[] arr = new int[3];  
    char ch = 'a';  
}
```



ערכם של איברי המערך שהוקצה הוא 0.

- מערך הוא עצם (אובייקט), נושא שילמד לעומק בהמשך.
- **הקצאת השטחים** עבור העצמים מתבצעים ב-**heap**.
- כלומר, איברי המערך נשמרים ברצף בזכרון שנקרא **heap**.
- משתנים רגילים, והפניות לעצמים נמצאים ב-**stack**, שזה איזור שונה בזכרון.
- משתנים רגילים, פרמטרים והפניות מוקצים ב-**stack** עבור כל פעולה מחדש.

אתחול מערכים : דוגמת הזכרון

```
int[] numbers = new int[] { 5, 3, 1 };  
char[] letters = new char[] { 'm', 'A', 'k' };
```

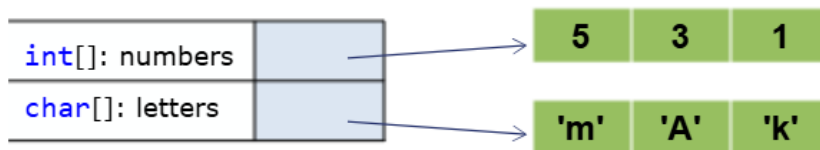
```
int[] numbers = { 5, 3, 1 };  
char[] letters = { 'm', 'A', 'k' };
```

- עבור המערכים הבאים :

ניתן לאתחל את כל ערכי המערך ישירות בפקודה של הגדרת המערך.

או כך:

- הזיכרון יראה כך:



גישה לאיברי המערך

- כדי שניתן יהיה להתייחס לכל איבר בנפרד ניתנו להם אינדקסים.
- האיבר הראשון בעל אינדקס 0, השני בעל אינדקס 1 והאחרון עם אינדקס `size-1`.

```
int[] numbers = new int[3];  
numbers[0] = 25;  
numbers[0]++;
```



- הפניה לאיבר מסוים במערך היא ע"י [] כאשר בתוך הסוגריים יהיה האינדקס של האיבר אליו נרצה לגשת.
- פניה לאיבר במערך היא כפניה למשתנה מטיפוס המערך.
- ההתייחסות ל-`numbers[0]` היא כמו ההתייחסות לכל משתנה `int` רגיל.

גישה לאיברי המערך

- כדי שניתן יהיה להתייחס לכל איבר בנפרד ניתנו להם אינדקסים.
- האיבר הראשון בעל אינדקס 0, השני בעל אינדקס 1 והאחרון עם אינדקס `size-1`.

```
int[] numbers = new int[3];  
numbers[0] = 25;  
numbers[0]++;
```



- הפניה לאיבר מסוים במערך היא ע"י [] כאשר בתוך הסוגריים יהיה האינדקס של האיבר אליו נרצה לגשת.
- פניה לאיבר במערך היא כפניה למשתנה מטיפוס המערך.
- ההתייחסות ל-`numbers[0]` היא כמו ההתייחסות לכל משתנה `int` רגיל.

גישה לאיברי המערך

- כדי שניתן יהיה להתייחס לכל איבר בנפרד ניתנו להם אינדקסים.
 - האיבר הראשון בעל אינדקס 0, השני בעל אינדקס 1 והאחרון עם אינדקס `size-1`.

```
int[] numbers = new int[3];  
numbers[0] = 25;  
numbers[0]++;
```



- הפניה לאיבר מסוים במערך היא ע"י [] כאשר בתוך הסוגריים יהיה האינדקס של האיבר אליו נרצה לגשת.
- פניה לאיבר במערך היא כפניה למשתנה מטיפוס המערך.
 - ההתייחסות ל-`numbers[0]` היא כמו ההתייחסות לכל משתנה `int` רגיל.

גישה לאיברי המערך : דוגמה

```
static void UsingIndexes()  
{
```

```
    int[] numbers = new int[3];
```

```
    numbers[0] = 4;
```

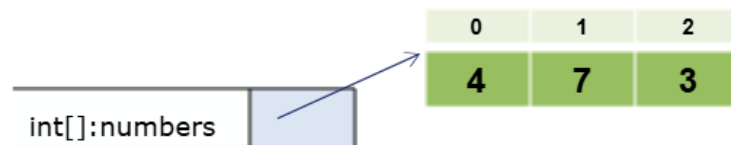
```
    numbers[1] = 7;
```

```
    numbers[2] = 3;
```

```
    Console.WriteLine("The values in the array: "
```

```
                        + numbers[0] + " " + numbers[1] + " " + numbers[2]);
```

```
}
```



The values in the array: 4 7 3

מספר האיברים במערך : Length

- למערך ישנו מאפיין Length המעיד על גודלו :
- Length הוא מאפיין ולא פעולה, לכן נפנה אליו ללא שימוש בסוגריים.

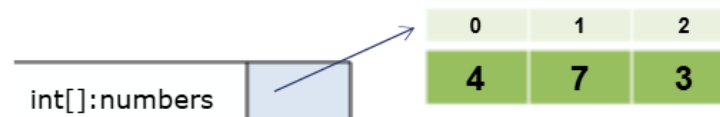
```
static void ArrayInMemory()  
{  
    char[] letters = {'m', 'A', 'k'};  
    Console.WriteLine("Array has " + letters.Length + " values");  
}
```

Array has 3 values

גישה לאיברי המערך : לולאות

- עבודה עם איברי מערך היא עבודה זהה על כל איבריו, לכן נשתמש בלולאות.

```
static void ArraysAndLoops()
{
    int[] numbers = new int[3];
```



```
    Console.WriteLine("Enter " + numbers.Length + " numbers: ");
```

```
    for (int i = 0; i < numbers.Length; i++)
        numbers[i] = int.Parse(Console.ReadLine());
```

בעבודה עם לולאות העוברות על מערכים, ברוב המקרים, האינדקס יתחיל מ-0.

```
    Console.Write("The numbers are: ");
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
```

```
}
```

Enter 3 numbers:

4

7

3

The numbers are: 4 7 3

האינדקס שאיתו פונים לאיבר במערך יכול להיות:

1. מספר שלם (כמו בדוגמה הקודמת).
2. משתנה (כמו בדוגמה זו).
3. ערך של ביטוי, למשל: `arr[i+2]`.

גישה לכל איברי המערך באמצעות לולאות :

הדפסת ערכי המערך מהסוף להתחלה

```
static void PrintBackwards()
```

```
{
```

```
    int[] numbers = new int[3];
```

```
    Console.WriteLine("Enter " + numbers.Length + " numbers: ");
```

```
    for (int i = 0; i < numbers.Length; i++)
```

```
        numbers[i] = int.Parse(Console.ReadLine());
```

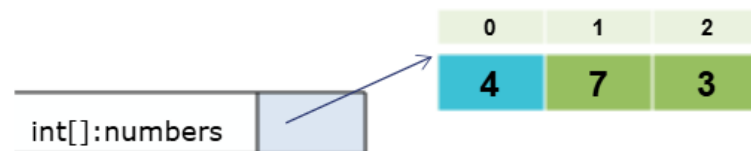
```
    Console.Write("The numbers are: ");
```

```
i=-1    for (int i = numbers.Length - 1; i >= 0; i--)
```

```
        Console.Write(numbers[i] + " ");
```

```
    Console.WriteLine();
```

```
}
```



Enter 3 numbers:

4

7

3

The numbers are: 3 7 4

חריגה מגבולות המערך

- כדי לגשת לאחד מאיברי מערך בגודל N ניגש עם אינדקסים 0...N-1.
- כאשר ננסה לפנות לאינדקס שאינו בגבולות המערך אנו למעשה מנסים לגשת לתא בזיכרון שהוא מעבר לשטח שהוקצה למערך, וזו גישה לא חוקית לשטח בזיכרון. מכיון שאיננו יודעים מה יש בשטח זה ומה השפעתו.

```
86. static void IndexOutOfBounds()  
87. {  
88.     int[] numbers = new int[3];  
89.     numbers[5] = 8;  
90. }  
91.  
92. static void Main(string[] args)  
93. {  
94.     IndexOutOfBounds();  
95. }
```

Unhandled Exception: System.IndexOutOfRangeException:

Index was outside the bounds of the array.

at arrays.Program.IndexOutOfBounds() in C:\Code\arrays\arrays\Program.cs:line 89

at arrays.Program.Main(String[] args) in C:\Users\Code\arrays\arrays\Program.cs:line 94

חריגה מגבולות המערך : פניה לאינדקס אינטראקטיבי

- הקומפיילר **אינו מתריע** על ניסיון פניה לתא שאינו בגבולות המערך.
- מכיון שהאינדקס של התא ייקבל ערך רק בזמן ריצה (לא בזמן קומפילציה).

```
static void IndexOutOfBounds()  
{  
    int[] arr = new int[4];  
  
    Console.Write("Enter index: ");  
    int i = int.Parse(Console.ReadLine());  
    arr[i] = 7;  
    Console.WriteLine("Succeed!");  
}
```

```
Enter index: 2  
Succeed!
```

```
Enter index: 8  
  
Unhandled Exception: System.IndexOutOfRangeException  
Index was outside the bounds of the array.  
...
```

- במקרה של חריגה מגבולות המערך התוכנית תעוף עם חריגה `IndexOutOfRangeException`

זו אחריות המתכנת לוודא שפונים רק לתא בגבולות המערך !

חריגה מגבולות המערך : אחריות המתכנת לוודא תקינות

```
static void IndexOutOfBounds()
{
    int[] arr = new int[4];

    Console.Write("Enter index: ");
    int i = int.Parse(Console.ReadLine());
    if (i >= 0 && i < arr.Length)
    {
        arr[i] = 7;
        Console.WriteLine("Succeed!");
    }
    else
        Console.WriteLine("Invalid index!");
}
```

Enter index: 8
Invalid index!

Enter index: 2
Succeed!

נוודא שפונים רק לתא בגבולות המערך !

השמת מערכים

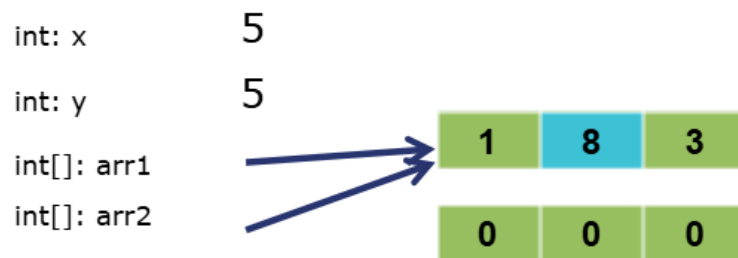
- כדי לבצע השמה בין משתנים פרימיטיביים (`int`, `double`, `char` וכד') מאותו הסוג אנו משתמשים באופרטור `=`.

- עבור מערכים, פעולה ההשמה מעתיקה את ההפניה.

- שני משתנים יכולים להכיל הפניה לאותו מערך.

```
static void Assignment()
{
    int x, y = 5;
    x = y;

    int[] arr1 = { 1, 2, 3 }, arr2 = new int[3];
    arr2 = arr1;
    arr1[1] = 8;
}
```



- העתקת ערכי המערכים תבוצע בעזרת לולאה, בה נעתיק את הערכים איבר-איבר.

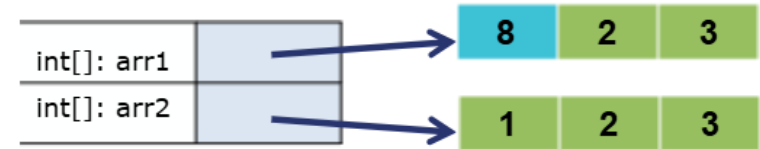
העתקת ערכי מערכים : דוגמה

```
static void CopyArray()
{
    int[] arr1 = { 1, 2, 3 }, arr2 = new int[3];

    Console.WriteLine("Elements in arr1 before: ");
    PrintArray (arr1);
    Console.WriteLine("Elements in arr2 before: ");
    PrintArray (arr2);

    for (int i = 0; i < arr2.Length; i++)
        arr2[i] = arr1[i];
    arr1[0] = 8;

    Console.WriteLine("\nElements in arr1 after: ");
    PrintArray (arr1);
    Console.WriteLine("Elements in arr2 after: ");
    PrintArray (arr2);
}
```



```
Elements in arr1 before: 1 2 3
Elements in arr2 before: 0 0 0
```

```
Elements in arr1 after: 8 2 3
Elements in arr2 after: 1 2 3
```

```
static void PrintArray(int[] arr)
{
    for (int i = 0; i < arr.Length; i++)
        Console.Write(arr [i] + " ");
    Console.WriteLine();
}
```

```

static void CountAboveAverage()
{
    int num1, num2, /*...*/num20, sum = 0;
    double average;
    int countAboveAverage = 0;

    Console.WriteLine("Please insert 20 numbers: ");
    num1 = int.Parse(Console.ReadLine());
    num2 = int.Parse(Console.ReadLine());
    /*...*/
    num20 = int.Parse(Console.ReadLine());

    sum = num1 + num2 /*+ ...*/ + num20;
    average = sum / 20.0;

    if (num1 > average)
        countAboveAverage++;
    if (num2 > average)
        countAboveAverage++;
    /*...*/
    if (num20 > average)
        countAboveAverage++;

    Console.WriteLine(countAboveAverage + " values are above the average " + average);
}

```

זוכרים?..

- תוכנית הקוראת 20 מספרים ומציגה כמה מספרים גדולים מהממוצע.
- עכשיו, נשתמש במערך ובלולאות.

התוכנית מסורבלת ומייגע לכתוב אותה, בייחוד כי יתכן גם שנרצה ממוצע של 100 מספרים, או אפילו יותר...

זוכרים?..

```
static void CountAboveAverage()
{
    int[] numbers = new int [100];
    int sum = 0;
    double average;
    int countAboveAverage = 0;

    Console.WriteLine("Please insert " + numbers.Length + " numbers:");
    for (int i = 0; i < numbers.Length; i++)
    {
        numbers[i] = int.Parse(Console.ReadLine());
        sum += numbers[i];
    }

    average = (double)sum / numbers.Length;

    for (int i = 0; i < numbers.Length; i++)
    {
        if (numbers[i] > average)
            countAboveAverage++;
    }
}
```

- תוכנית הקוראת 20 מספרים ומציגה כמה מספרים גדולים מהממוצע.
- עכשיו, נשתמש במערך ובלולאות.

התוכנית מסורבלת ומייגע לכתוב אותה, בייחוד כי יתכן גם שנרצה ממוצע של 100 מספרים, או אפילו יותר...

```
Console.WriteLine(countAboveAverage + " values are above the average " + average);
```

```
}
```

תזכורת : משמעות העברת פרמטר by value

```
static void IncNumber(int x)
{
    Console.WriteLine ("before increment: x=" + x);
    x++;
    Console.WriteLine ("after increment: x=" + x);
}
```

```
static void Main(string[] args)
{
    int num = 3;

    Console.WriteLine ("before function: num=" + num);
    IncNumber(num);

}
```

int: x	4
--------	---

הזיכרון של IncNumber

int: num	3
----------	---

הזיכרון של Main

```
before function: num=3
before increment: x=3
```

תזכורת : משמעות העברת פרמטר by value

```
static void IncNumber(int x)
{
    Console.WriteLine ("before increment: x=" + x);
    x++;
    Console.WriteLine ("after increment: x=" + x);
}
```

```
static void Main(string[] args)
{
    int num = 3;

    Console.WriteLine ("before function: num=" + num);
    IncNumber(num);
    Console.WriteLine ("after function: num=" + num);
}
```

במקרה של משתנים פשוטים:
העברת פרמטר לפעולה פרושה
יצירת עותק של הערך (by Value) (העברה by Value)
ולכן כל שינוי בו – לא משפיע על הערך המקורי.

int: num	3
----------	---

הזיכרון של Main

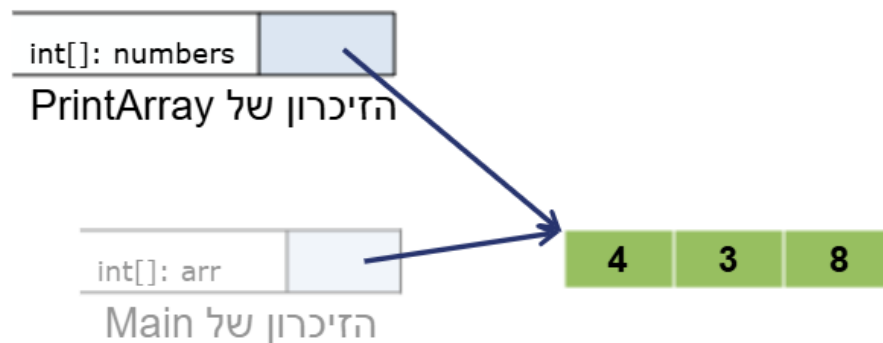
```
before function: num=3
before increment: x=3
after increment: x=4
after function: num=3
```

מערכים כפרמטר לפעולה

```
static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}

static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr); 4 3 8
}
}
```



מערכים כפרמטר לפעולה

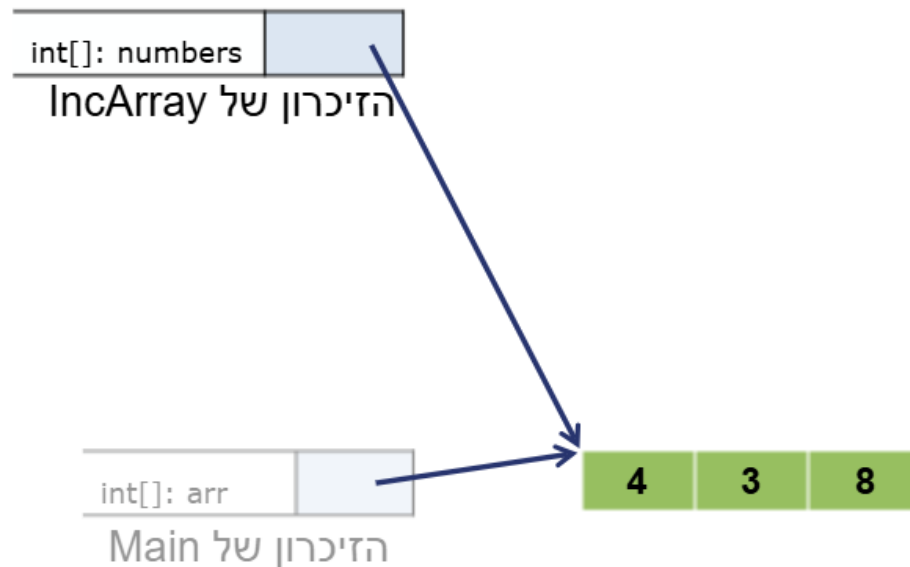
```
static void IncArray(int[] numbers)
{

}

static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}

static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr);
    IncArray(arr);
}
```



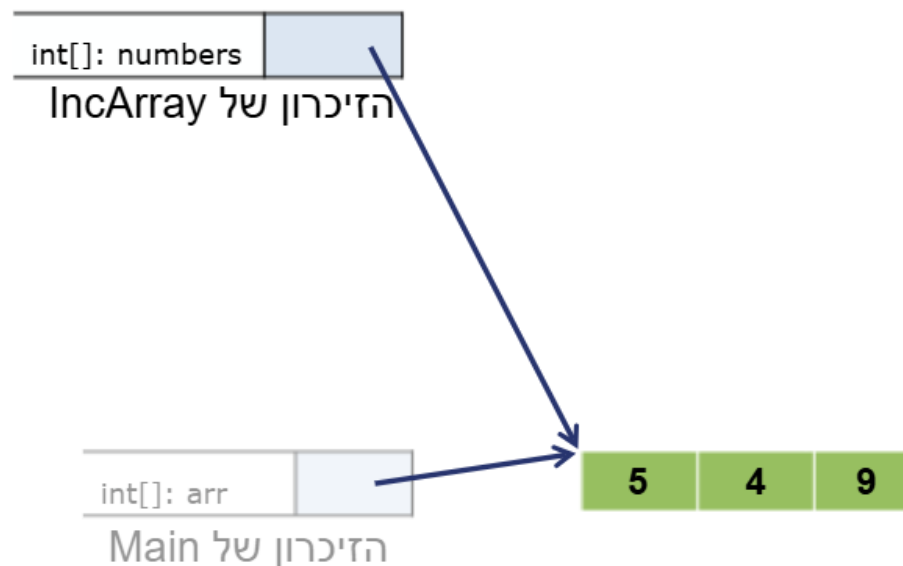
מערכים כפרמטר לפעולה

```
static void IncArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        numbers[i]++;
}
```

```
static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}
```

```
static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr);
    IncArray(arr);
}
```



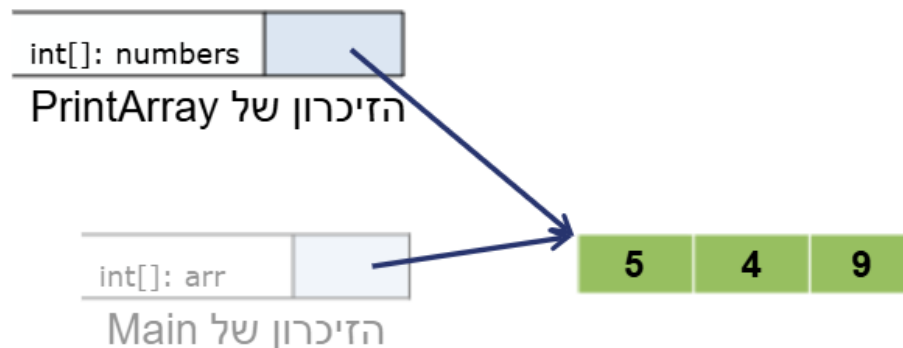
מערכים כפרמטר לפעולה

```
static void IncArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        numbers[i]++;
}
```

```
static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}
```

```
static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr);
    IncArray(arr);
    PrintArray(arr);
}
```



מערכים כפרמטר לפעולה

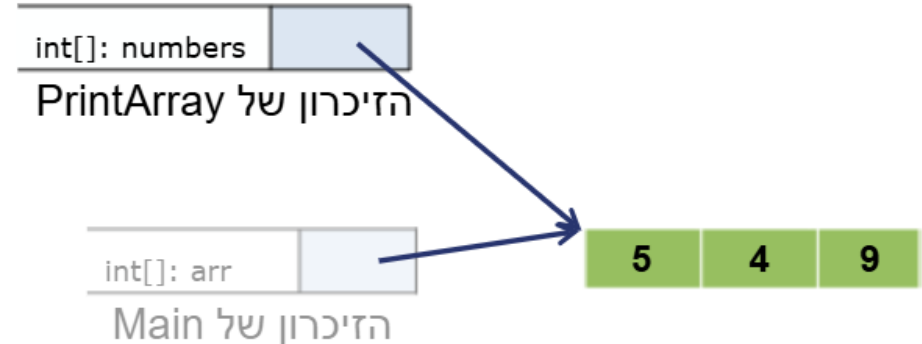
במקרה של משתנים מורכבים (עצמים):
העברת פרמטר לפעולה פרושה העברת ההפניה
לעצם (by reference).
ולכן כל שינוי בעצם משפיע על העצם המקורי.
ההפניה המועברת הינה הפניה נוספת לאותו העצם.

```
static void IncArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        numbers[i]++;
}

static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}

static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr); 4 3 8
    IncArray(arr);
    PrintArray(arr); 5 4 9
}
```



מערכים כפרמטר לפעולה

במקרה של משתנים מורכבים (עצמים):
העברת פרמטר לפעולה פרושה העברת ההפניה לעצם (by reference).
ולכן כל שינוי בעצם משפיע על העצם המקורי.
ההפניה המועברת הינה הפניה נוספת לאותו העצם.

```
static void IncArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        numbers[i]++;
}
```

```
static void PrintArray(int[] numbers)
{
    for (int i = 0; i < numbers.Length; i++)
        Console.Write(numbers[i] + " ");
    Console.WriteLine();
}
```

```
static void Main(string[] args)
{
    int[] arr = { 4, 3, 8 };

    PrintArray(arr);
    IncArray(arr);
    PrintArray(arr);
}
```

4 3 8
5 4 9



מערכים כפרמטר לפעולה : סיכום

- ראינו כי מערך (עצם) הוא משתנה המורכב מהפניה ומשטח זכרון שהוקצה לו.
- בהעברת משתנה פשוט כפרמטר לפעולה מועבר רק עותק של המשתנה (**by value**) כל שינוי בעותק הוא מקומי לפעולה עצמה ולא משפיע על הערך המקורי.
- בהעברת משתנה מורכב כפרמטר לפעולה מועברת ההפניה לעצם (**by reference**) כל שינוי בעצם, מתבצע תוך שימוש בהפניה, ולכן הוא משפיע על הערך המקורי של העצם.
- בהעברת משתנה מורכב כפרמטר לפעולה מועברת ההפניה לעצם (**by reference**), ההפניה מועברת כעותק ולכן אם משנים אותה, למשל במידה וייצרנו מערך אחר, יש להחזיר את ההפניה מהפעולה, כלומר יש להחזיר את המערך !
- הנושא יוסבר לעומק כאשר נלמד על עצמים.

```
static int[] CreateAndReadArr(int size)
{
    int[] arr = new int[size];
    for (int i=0 ; i < arr.Length ; i++)
        arr[i] = int.Parse(Console.ReadLine());

    return arr;
}
```

```
static void Main(string[] args)
{
    Console.Write("How many elements? ");
    int size = int.Parse(Console.ReadLine());

    int[] numbers = CreateAndReadArr(size);

    PrintArray(numbers);
}
```

החזרת מערך מפעולה

int: size	4
int[]: numbers	

הזיכרון של Main

החזרת מערך מפעולה

```
static int[] CreateAndReadArr(int size)
{
    int[] arr = new int[size];
    for (int i=0 ; i < arr.Length ; i++)
        arr[i] = int.Parse(Console.ReadLine());

    return arr;
}
```

```
static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());

    int[] numbers = CreateAndReadArr(size);

    PrintArray(numbers);
}
```

int: size	4
int[]: arr	

הזיכרון של
CreateAndReadArr

0	1	2	3
62	29	17	84

int: size	4
int[]: numbes	

הזיכרון של Main

החזרת מערך מפעולה

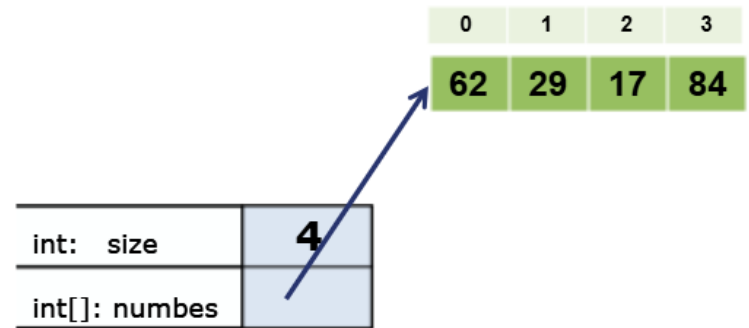
```
static int[] CreateAndReadArr(int size)
{
    int[] arr = new int[size];
    for (int i=0 ; i < arr.Length ; i++)
        arr[i] = int.Parse(Console.ReadLine());

    return arr;
}
```

```
static void Main(string[] args)
{
    Console.Write("How many elements? ");
    int size = int.Parse(Console.ReadLine());

    int[] numbers = CreateAndReadArr(size);

    PrintArray(numbers);
}
```



הזיכרון של Main

החזרת מערך מפעולה

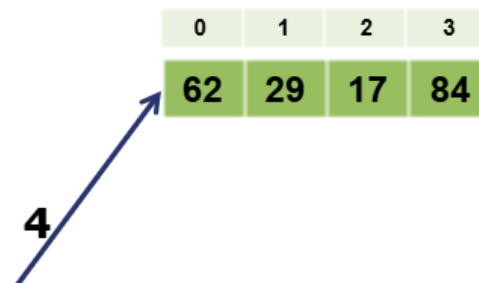
```
static int[] ResizeArr(int[] arr, int size)
{
    int[] tmp = new int[size]; // יצירת מערך חדש
    for (int i=0 ; i < size ; i++)
        tmp[i] = arr[i]; // העתקת הערכים

    return tmp; // החזרת המערך החדש
}
```

```
static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());
    int[] numbers = CreateAndReadArr; // הפעולה מקודם
```

```
    Console.WriteLine("What is the new elements amount? ");
    int newSize = int.Parse(Console.ReadLine());
    numbers = ResizeArr(numbers, newSize); // הצבה של ההפניה במשתנה המקורי
```

```
    PrintArray(numbers);
}
```



int: newSize ???
הזיכרון של Main

החזרת מערך מפעולה

```
static int[] ResizeArr(int[] arr, int size)
{
    int[] tmp = new int[size]; // יצירת מערך חדש
    for (int i=0 ; i < size ; i++)
        tmp[i] = arr[i]; // העתקת הערכים

    return tmp; // החזרת המערך החדש
}
```

```
static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());
    int[] numbers = CreateAndReadArr; // הפעולה מקודם

    Console.WriteLine("What is the new elements amount? ");
    int newSize = int.Parse(Console.ReadLine());
    numbers = ResizeArr(numbers, newSize); // הצבה של ההפניה במשתנה המקורי

    PrintArray(numbers);
}
```

int: size **3**
int[]: arr
int[]: tmp **???**
הזיכרון של ResizeArr



0	1	2	3
62	29	17	84

3
הזיכרון של Main

החזרת מערך מפעולה

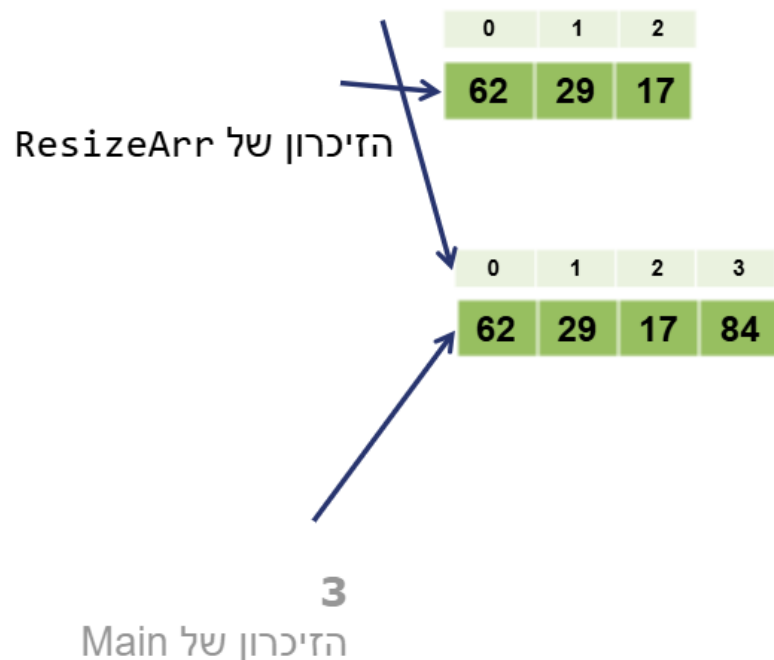
```
static int[] ResizeArr(int[] arr, int size)
{
    int[] tmp = new int[size]; // יצירת מערך חדש
    for (int i=0 ; i < size ; i++)
        tmp[i] = arr[i]; // העתקת הערכים

    return tmp; // החזרת המערך החדש
}

static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());
    int[] numbers = CreateAndReadArr; // הפעולה מקודם

    Console.WriteLine("What is the new elements amount? ");
    int newSize = int.Parse(Console.ReadLine());
    numbers = ResizeArr(numbers, newSize); // הצבה של ההפניה במשתנה המקורי

    PrintArray(numbers);
}
```



החזרת מערך מפעולה

```
static int[] ResizeArr(int[] arr, int size)
{
    int[] tmp = new int[size]; // יצירת מערך חדש
    for (int i=0 ; i < size ; i++)
        tmp[i] = arr[i]; // העתקת הערכים

    return tmp; // החזרת המערך החדש
}

static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());
    int[] numbers = CreateAndReadArr; // הפעולה מקודם

    Console.WriteLine("What is the new elements amount? ");
    int newSize = int.Parse(Console.ReadLine());
    numbers = ResizeArr(numbers, newSize); // הצבה של ההפניה במשתנה המקורי

    PrintArray(numbers);
}
```



החזרת מערך מפעולה

```
static int[] ResizeArr(int[] arr, int size)
{
    int[] tmp = new int[size]; //ש
    for (int i=0 ; i < size ; i++)
        tmp[i] = arr[i]; //ת הערכים

    return tmp; //החזרת המערך החדש
}
```

במקרה של משתנים מורכבים (עצמים):
העברת פרמטר לפעולה פרושה
יצירת עותק של ההפניה לעצם
ולכן כל שינוי בהפניה, למשל ביצירת מערך חדש,
לא משפיע על ההפניה המקורית ולכן **חייבים**
להחזיר אותה - להחזיר את המערך.

```
static void Main(string[] args)
{
    Console.WriteLine("How many elements? ");
    int size = int.Parse(Console.ReadLine());
    int[] numbers = CreateAndReadArr; //הפעולה מקודם
```

```
    Console.WriteLine("What is the new elements amount? ");
    int newSize = int.Parse(Console.ReadLine());
    numbers = ResizeArr(numbers, newSize); //הצבה של ההפניה במשתנה המקורי;
```

```
    PrintArray(numbers);
}
```

0	1	2
62	29	17

0	1	2	3
62	29	17	84

3

הזיכרון של Main

דוגמה : הדפסת היסטוגרמה של ערכי המערך

```
static void PrintHistogram(int[] arr)
{
    Console.WriteLine("There are " + arr.Length + " numbers in the array:");
    for (int i=0 ;i < arr.Length ;i++ )
    {
        Console.Write(arr[i] + ": ");
        for (int j=0 ; j < arr[i] ; j++)
            Console.Write("*");
        Console.WriteLine();
    }
}
```

There are 4 numbers in the array:
4: ****
3: ***
2: **
7: *****

4: ****
3: ***
2: **
7: *****

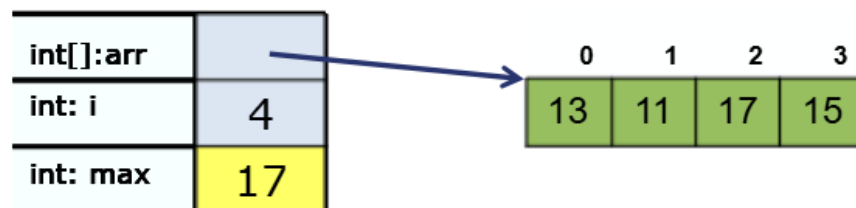
int[:arr		0	1	2	3
int: i	4	4	3	2	7
int: j					

מציאת הערך המקסימלי

נאתחל את המקסימום באיבר הראשון של המערך ונתחיל לחפש איבר גדול יותר, החל מהאיבר השני

```
static int MaxValue(int[] arr)
{
    int max = arr[0];
    for ( int i=1 ; i < arr.Length ; i++ )
    {
        if (arr[i] > max)
            max = arr[i];
    }

    return max;
}
```



מציאת האינדקס שבתוכו הערך המקסימלי

```
static int IndexWithMaxValue(int[] arr)
{
```

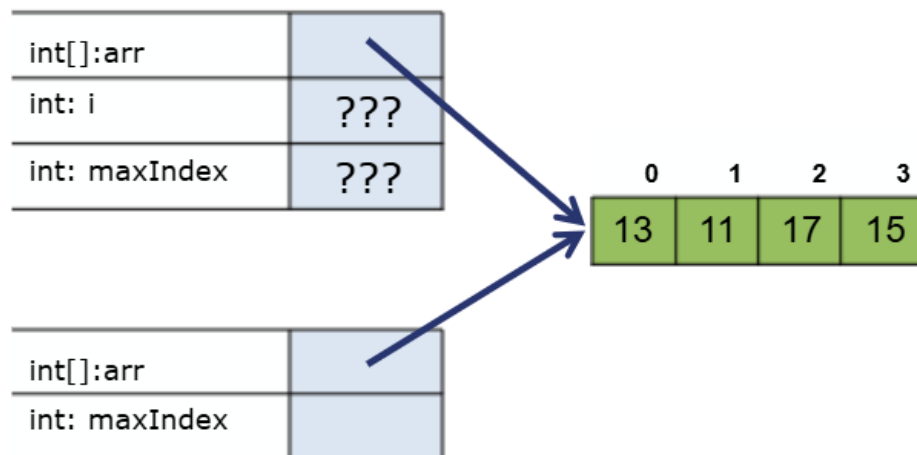
```
}
```

```
static void Main(string[] args)
{
```

```
    int[] arr = { 13, 11, 17, 15 };
```

```
    int maxIndex = IndexWithMaxValue(arr);
```

```
}
```



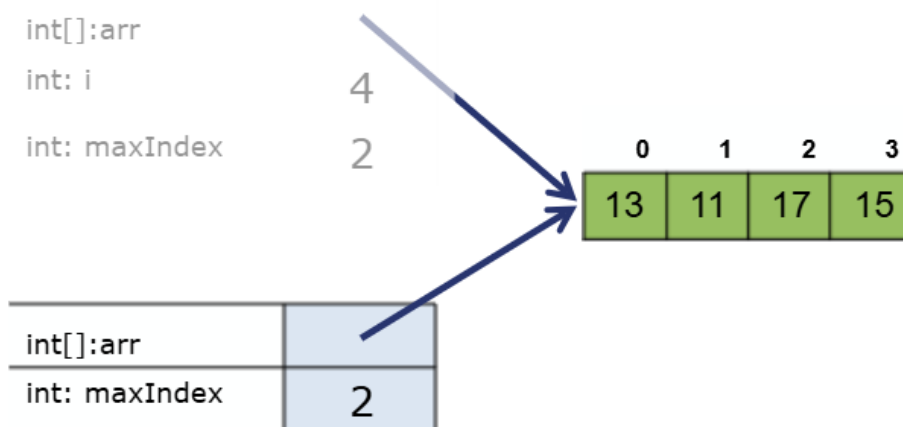
מציאת האינדקס שבתוכו הערך המקסימלי

```
static int IndexWithMaxValue(int[] arr)
{
    int maxIndex = 0;
    for (int i=1 ; i < arr.Length ; i++)
    {
        if (arr[i] > arr[maxIndex])
            maxIndex = i;
    }
    return maxIndex;
}

static void Main(string[] args)
{
    int[] arr = { 13, 11, 17, 15 };

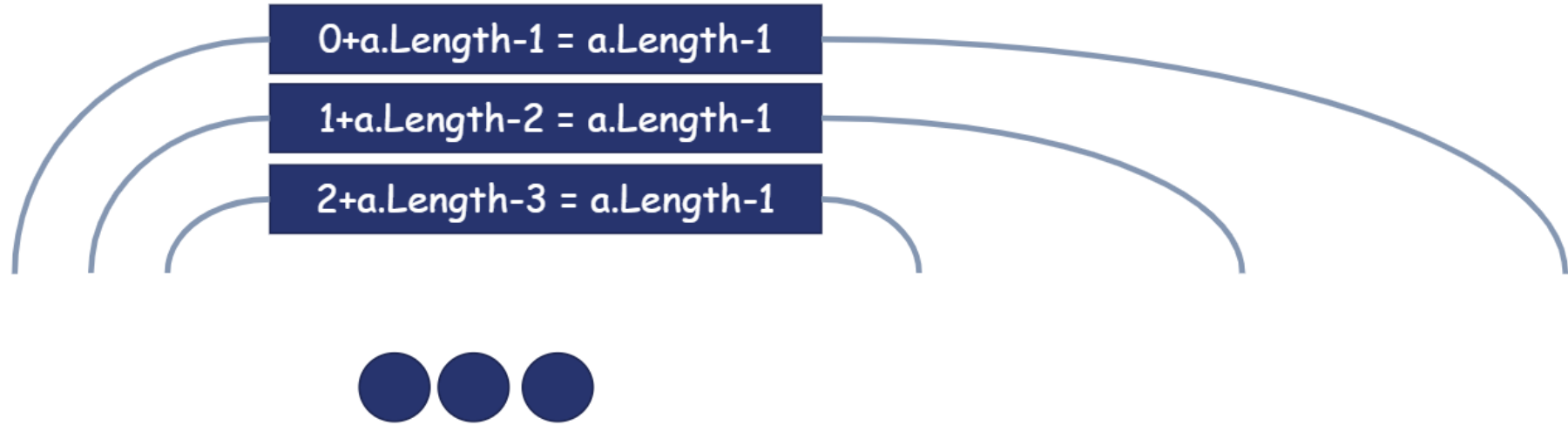
    int maxIndex = IndexWithMaxValue(arr);
    Console.WriteLine("The max is " + arr[maxIndex] + " at index " + maxIndex);
}
```

נאתחל את המיקום של המקסימום ל-0.
ונתחיל לחפש מיקום בו ערך האיבר גדול יותר.



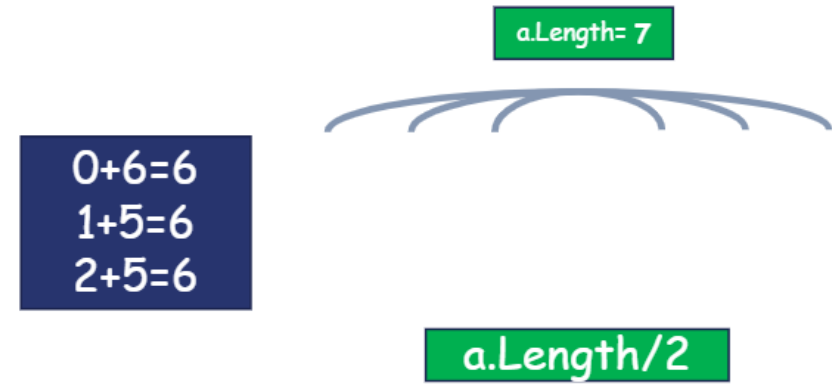
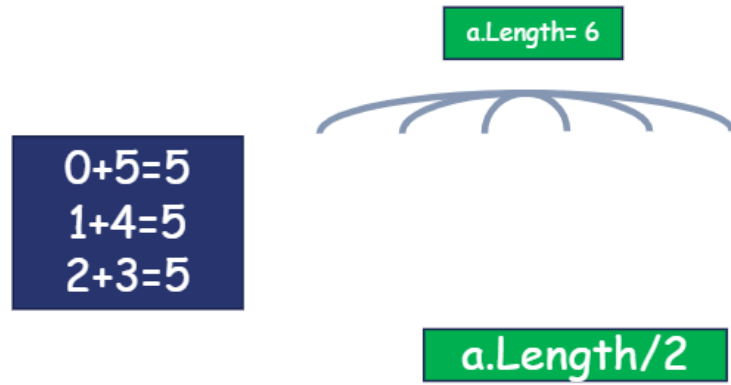
The max is 17 at index 2

סכום אינדקסים של תאים סימטריים : $\text{Length} - 1$



התא האחרון

היכן נמצא התא האמצעי ?



הפיכת סדר האיברים במערך

- נכתוב פעולה המקבלת מערך ומחזירה מערך חדש שסדר ערכיו הפוך.



```
static void Main(string[] args)
{
    int[] arr = { 10, 20, 30, 40 };
    int[] res;

    PrintArray(arr);
    res = CreateReversedArr(arr);
    PrintArray(res);
}
```

```
10 20 30 40
40 30 20 10
```

```
static int[] CreateReversedArr(int[] arr)
{
    int[] res = new int[arr.Length];

    i=3 for (int i = 0; i < res.Length; i++)
        res[i] = arr[arr.Length-1-i];
    return res;
}
```



מערך סימטרי (פלינדרום)

- הגדרה: מערך פלינדרום הוא מערך שאם נקרא את ערכיו משמאל לימין או מימין לשמאל נקבל ערכים בסדר זהה.



דוגמה : האם המערך הוא פלינדרום ?

```
static bool AreEqualArrays(int[] arr1, int[] arr2)
{
    if (arr1.Length != arr2.Length)
        return false;

    for (int i=0; i < arr1.Length; i++)
    {
        if (arr1[i] != arr2[i])
            return false;
    }
    return true;
}

static bool IsPalindrom(int[] arr)
{
    int[] revArr = CreateReversedArr(arr);
    return AreEqualArrays(arr, revArr);
}
```

- אם ערכי המערך וערכי המערך בסדר הפוך זהים, המערך הוא פלינדרום.

0	1	2	3	4
8	5	2	5	8

0	1	2	3	4
8	5	2	5	8

✗ אין אפשרות להציג את התמונה.

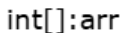
0	1	2	3	4
8	5	2	7	8

0	1	2	3	4
8	7	2	5	8

✗ אין אפשרות להציג את התמונה.

© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved.

}



```
int: left
```

```
int: right
```

```
bool: isSymertic    true
```



right

isSymetric = true

הבדיקה מתבצעת **כל עוד** לא מצאנו בעיה
`while (... && isSymetric)`

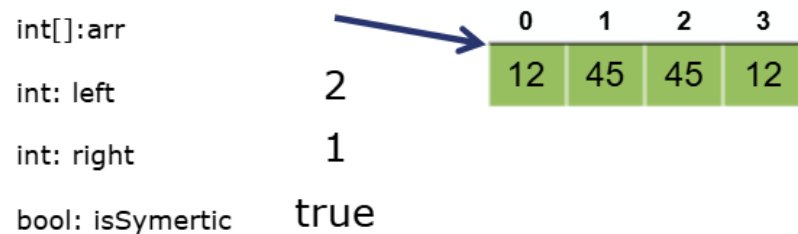
דוגמה 2: האם המערך הוא פלינדרום ?

```
static bool IsSymetricArray(int[] arr)
{
    int left, right;
    bool isSymetric = true;

    for (left=0, right=arr.Length-1; left < right && isSymetric ; left++, right--)
    {
        if (arr[left] != arr[right])
            isSymetric = false;
    }

    return isSymetric;
}
```

int[]:arr		0	1	2	3
int: left	2	12	45	45	12
int: right	1				
bool: isSymertic	true				



דוגמה 3 : האם המערך הוא פלינדרום ?

```
static bool IsSymetricArray(int[] arr)
{
    bool isSymetric = true;

    for (int i=0; i < arr.Length/2 && isSymetric ; i++)
    {
        isSymetric = (arr[i] != arr[arr.Length-1-i]);
    }

    return isSymetric;
}
```

		arr.Length = 5				
int[]:arr		0	1	2	3	4
bool: isSymertic	true	12	45	45	22	12

מספיק לעבור עד חצי המערך וכל עוד סימטרי (isSymetric),
ובכל פעם - לחשב, בצורה סימטרית, את מיקום האיברים להשוואה.

דוגמה 3 : האם המערך הוא פלינדרום ?

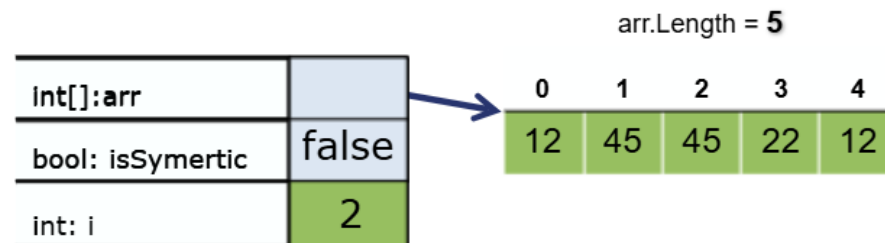
```
static bool IsSymetricArray(int[] arr)
{
```

```
    bool isSymetric = true;
```

```
    for (int i=0; i < arr.Length/2 && isSymetric ; i++)
    {
        isSymetric = (arr[i] != arr[arr.Length-1-i]);
    }
```

```
    return isSymetric;
```

```
}
```



מספיק לעבור עד חצי המערך וכל עוד סימטרי (isSymetric),
ובכל פעם - לחשב, בצורה סימטרית, את מיקום האיברים להשוואה.

מה עושה הקוד הבא ?

```
static int What(int[] arr)
{
    int count = 1;

    for (int i = 1; i < arr.Length ; i++)
    {
        if (arr[i] == arr[0])
            count++;
    }

    if (count == arr.Length)
        return 1;
    else
        return 0;
}
```

קוד חביב, אך אפשר לכתוב אותו יותר קצר ופחות מסורבל.

ספירת מספר האיברים שערכיהם זהים לערך האיבר הראשון.

אם התנאי מתקיים, משמע כל ערכי המערך זהים והפעולה תחזיר 1, אחרת תחזיר 0.

מה עושה הקוד הבא ?

```
static int What(int[] arr)
{
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] == arr[0])
        {
        }
        else
        {
            return 0;
        }
    }
    return 1;
}
```

צורת הכתיבה הזו עובדת, אבל עקומה !
לא נכתוב תנאי עם גוף ריק !

הפעולה מחזירה 0 במידה וקיים ערך שאינו זהה לערך האיבר הראשון ויוצאת, כלומר: התוכנית מחזירה 1 אם ערכי כל האיברים זהים, אחרת מחזירה 0.

שדרוג הקוד הקודם

```
static int What(int[] arr)
{
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] == arr[0])
        {
        }
        else
        {
            return 0;
        }
    }
    return 1;
}
```

```
if (arr[i] != arr[0])
{
    return 0;
}
```

לא נכתוב תנאים ריקים !
פשוט
נהפוך את התנאי

ניתן להשתמש גם בבדיקת שונה ($\neq$)
ולא רק בבדיקת שוויון ($=$)

הקוד המתוקן

```
static int What(int[] arr)
{
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] != arr[0])
        {
            return 0;
        }
    }
    return 1;
}
```

זהירות! טעות נפוצה – כתיבת else מיותר ושגוי

```
static int What(int[] arr)
{
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] != arr[0])
        {
            return 0;
        }
        else
        {
            return 1;
        }
    }
    return 1;
}
```

באופן כתיבה זה תוחזר התשובה
כבר אחרי הסיבוב הראשון של הלולאה,
ולמעשה ייבדק רק הזוג הראשון במערך.

אין מניעה לכתוב if בלי else !

הקוד המתקן – שימוש במשתנה

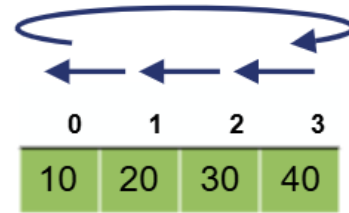
כדי להימנע מטעויות כאלה
נחזיר ערך – פעם אחת ורק בסוף הפעולה !

```
static int What(int[] arr)
{
    int returnValue = 1;    // ערך תקין להחזרה
    for (int i = 1; i < arr.Length && returnValue == 0 ; i++)
    {
        if (arr[i] != arr[0]) // האם משהו לא תקין ?
        {
            returnValue = 0;
        }
    }
    return returnValue;
}
```

נחזיק **משתנה** שיכיל את **הערך התקין** עבור ההחזרה
ונשנה אותו **רק** במידה וגילינו ש**משהו לא תקין**.
כמו-כן:
ניתן להמשיך בלולאה **רק** במידה ובמשתנה יש **ערך תקין**.

סיבוב מערך שמאלה

- נגדיר "**סיבוב מערך שמאלה**" כהעברת הערך הראשון להיות האחרון, וכל כל איבר מתקדם איבר אחד לאינדקס אחד יותר קטן



- דוגמה:
המערך

0	1	2	3
20	30	40	10

- אחרי סיבוב יתעדכן להיות

- כתבו פעולה המקבלת מערך ומסובבת אותו סיבוב אחד
- שימו לב שאין לדרוס ערך לפני הטיפול בו !

סיבוב מערך שמאלה : אסטרטגיית הפתרון

- נשמור את ערך האיבר הראשון במשתנה עזר.
- נעתיק מיקום אחד אחורה כל איבר מהשני ועד האחרון.
- נעתיק לאיבר האחרון את הערך ששמרנו במשתנה העזר.

0	1	2	3		
20	30	40	10	temp	10

סיבוב מערך שמאלה : פתרון

- נשמור את ערך האיבר הראשון במשתנה עזר.
- נעתיק מיקום אחד אחורה כל איבר מהשני ועד האחרון.
- נעתיק לאיבר האחרון את הערך ששמרנו במשתנה העזר.

```
static void RotateArray(int[] arr)
{
    int temp = arr[0];
    for (int i = 1; i < arr.Length ; i++)
    {
        arr[i-1] = arr[i];
    }
    arr[arr.Length - 1] = temp;
}
```

0	1	2	3
20	30	40	10

temp = 10

דוגמה : הדפסת התווים השונים במערך ומספרם

```
The letters are:  
a 0 ? T T 0 $ a T x  
The different letters are:  
a 0 ? T $ x  
There are 6 unique letters
```

- כתבו תוכנית המקבלת מערך תווים :

- הדפיסו למסך את התווים שהוקלדו אך ללא חזרות.

- החזירו את מספר התווים השונים.

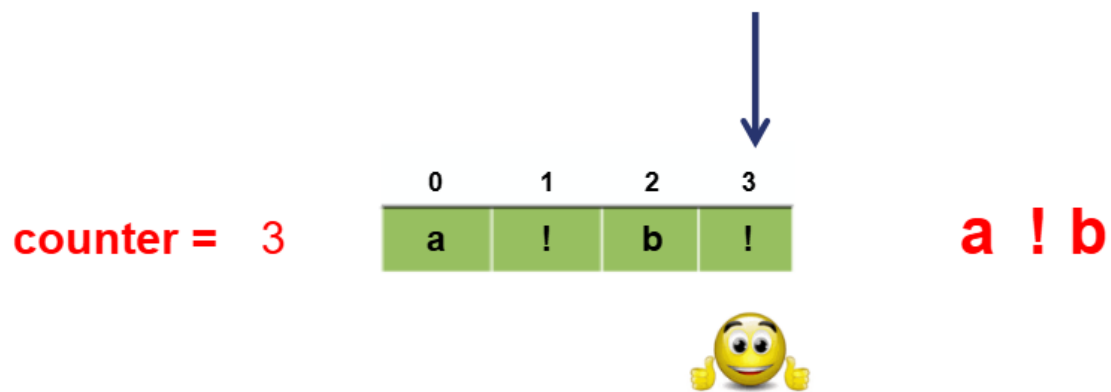
נגדיר 2 פעולות עזר: אחת לכל משימה

```
static void Main(string[] args)  
{  
    char[] letters = { 'a', '0', '?', 'T', 'T', '0', '$', 'a', 'T', 'x' };  
  
    Console.WriteLine("The letters are: ");  
    PrintLetters(letters);  
    Console.WriteLine("There are " + CountAndShowUnique(letters) + " unique letters");  
}
```

הדפסת התווים השונים במערך ומספרם :

אסטרטגיית הפתרון

- נגדיר **counter** לספירת התווים השונים.
- הרעיון:
 - עבור כל תו נבדוק אם הוא כבר הופיע בתווים שלפניו.
 - במידה ולא נגדיל את ה- **counter** ונדפיס את התו.
- בסוף נחזיר את ה-**counter**.



הדפסת התווים השונים במערך ומספרם : הקוד

- נגדיר פעולת עזר המקבלת מערך ומיקום (אינדקס) ומחזירה אמת אם האיבר במיקום שהועבר כפרמטר כבר הופיע במערך קודם, ושקר אחרת.

```
static bool HadAppearBefore(char[] arr, int index)
{
    for (int i=0; i < index; i++)
    {
        if (arr[index] == arr[i])
            return true;
    }
    return false;
}
```

0	1	2	3
a	!	b	!

index	i	i<index	arr[index]==arr[i]	return
2				
	0	true	false	
	1	true	false	
	2	false		
				false

הדפסת התווים השונים במערך ומספרם : הקוד

- נגדיר פעולת עזר המקבלת מערך ומיקום (אינדקס) ומחזירה אמת אם האיבר במיקום שהועבר כפרמטר כבר הופיע במערך קודם, ושקר אחרת.

```
static bool HadAppearBefore(char[] arr, int index)
{
    for (int i=0; i < index; i++)
    {
        if (arr[index] == arr[i])
            return true;
    }
    return false;
}
```

0	1	2	3
a	!	b	!

index	i	i<index	arr[index]==arr[i]	return
3				
	0	true	false	
	1	true	true	true

הדפסת התווים השונים במערך ומספרם : הקוד

```
static int CountAndShowUnique(char[] arr)
{
    int count = 0;

    Console.WriteLine("The different letters are:");

    for (int i = 0; i < arr.Length; i++)
    {
        if (!HadAppearBefore(arr, i))
        {
            Console.Write(arr[i] + " ");
            count++;
        }
    }
    Console.WriteLine();
    return count;
}
```

0	1	2	3
a	!	b	!

count	i	i<Length	!HadAppearBefore(arr,i)	output	return
0					
1	0	true	true	a	
2	1	true	true	!	
3	2	true	true	b	
	3	true	false		
	4	false			
					3

מיזוג מערכים ממוינים

מערכ ממויין הוא מערך שאיבריו מסודרים
מהערך הקטן לגדול, או מהערך הגדול לקטן

1. כל עוד יש איברים בשני המערכים :

1.1 אם האיבר הנוכחי במערך הראשון קטן/שווה מהאיבר הנוכחי במערך השני:

1.1.1 נעתיק איבר נוכחי מהמערך הראשון לנוכחי בשלישי

1.1.2 נקדם את מיקום האיבר הנוכחי במערך הראשון

1.2 אחרת:

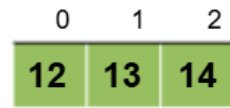
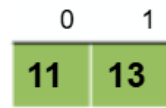
1.2.1 נעתיק איבר נוכחי מהמערך השני לנוכחי בשלישי

1.2.2 נקדם את מיקום האיבר הנוכחי במערך השני

1.3 נקדם את מיקום האיבר במערך השלישי

2. נעתיק את כל מה שנותר מהמערך הראשון לשלישי
3. נעתיק את כל מה שנותר מהמערך השני לשלישי

{ רק אחד מהם יקרה...



מיזוג מערכים ממוינים : הקוד

```
static int[] MergeArrays(int[] arr1, int[] arr2)
{
    int[] res = new int[arr1.Length + arr2.Length];
    int i=0, j=0, m=0;

    while ( i < arr1.Length && j < arr2.Length )
    {
        if (arr1[i] <= arr2[j])
            res[m++] = arr1[i++];
        else
            res[m++] = arr2[j++];
    }

    while ( i < arr1.Length) // copy rest of arr1
        res[m++] = arr1[i++];

    while ( j < arr2.Length) // copy rest of arr2
        res[m++] = arr2[j++];

    return res;
}
```

i = 2 arr1:

0	1
11	13

j = 3 arr2:

0	1	2
12	13	14

m = 5 res:

0	1	2	3	4
11	12	13	13	14

גודל המערך הממוזג
הוא סכום הגדלים של מערכי המקור.

מיזוג מערכים ממוינים : הקוד – ללא משתנה עזר (אינדקס)

```
static int[] MergeArrays(int[] arr1, int[] arr2)
{
    int[] res = new int[arr1.Length + arr2.Length];
    int i=0, j=0;

    while ( i < arr1.Length2 && j < arr2.Length3 )
    {
        if (arr1[i] <= arr2[j])
            res[i+j] = arr1[i++];
        else
            res[i+j] = arr2[j++];
    }

    while ( i < arr1.Length) // copy rest of arr1
        res[i+j] = arr1[i++];

    while ( j < arr2.Length) // copy rest of arr2
        res[i+j] = arr2[j++];

    return res;
}
```

i = 2 arr1:

0	1
11	13

j = 3 arr2:

0	1	2
12	13	14

res:

0	1	2	3	4
11	12	13	13	14

האינדקס של המערך הממוזג
הוא סכום האינדקסים של מערכי המקור.

מערך מונים : **המורה של המדינה**

- על-מנת להחליט מי המנצח, נגדיר מערך שגודלו כמספר האפשרויות, כאשר כל מיקום ייצג מועמד, וערכו של כל מיקום יאותחל ב-0.
- למשל המיקום ה-0 ייצג את המורה גוגו, המיקום ה-1 את המורה מומו וכו'.
- נציג לקהל אפשרויות :
 - ✓ הקלד 1 לגוגו
 - ✓ הקלד 2 למומו
 - ✓ וכו'
- לבסוף נבדוק באיזה אינדקס יש את הערך הגדול ביותר.

0 (גוגו)	1 (מומו)	2 (יוני)	3 (קוקו)
1	2	1	0

מומו

דוגמה : המורה של המדינה - פלט

```
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 2
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 2
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 3
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 3
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: -1
Each teacher and number of votes:
gogo --> 0 votes
momo --> 4 votes
yoyo --> 2 votes
koko --> 2 votes
The chosen teacher is: momo
```

0 (גוגו)	1 (ממו)	2 (ייו)	3 (קוקו)
0	4	2	2

המורה של המדינה :

הקוד

```
static void Main(string[] args)
{
```

```
    string[] names = { "gogo", "momo", "yoyo", "koko" };
    int[] votes = new int[names.Length];
```

```
    InitializeCountArr(votes);
    ReadPreferences (votes, names);
```

```
    Console.WriteLine("Each teacher and number of votes:");
    PrintHowManyVotes(votes, names);
```

```
    PrintChosenTeachers (votes, names);
}
```

```
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 2
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 2
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 3
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 3
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 1
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: -1
```

Each teacher and number of votes:

gogo --> 0 votes

momo --> 4 votes

yoyo --> 2 votes

koko --> 2 votes

The chosen teacher is: momo

0	1	2	3
gogo	momo	yoyo	koko
0	4	2	2

המורה של המדינה :

הקוד

```
static void InitializeCountArr(int[] votes)
{
    for (int i = 0; i < votes.Length; i++)
        votes[i] = 0;
}
```



חובה לאפס מונה לפני השימוש בו
ולכן
נקפיד **תמיד** לאפס את אברי מערך המונים !

המורה של המדינה :

הקוד

```
static void ReadPreferences(int[] votes, string[] names)
```

```
{  
    const int EXIT = -1;  
    int choice;  
    bool fContinue = true;  
    while (fContinue)  
    {  
        Console.WriteLine("Choose ");  
        for (int i = 0; i < names.Length; i++)  
            Console.Write(i + " for " + names[i] + ", ");  
        Console.WriteLine(EXIT + " to exit:");  
  
        choice = int.Parse(Console.ReadLine());  
        if (choice >= 0 && choice < names.Length)  
            votes[choice]++;  
        else if (choice == EXIT)  
            fContinue = false;  
        else  
            Console.WriteLine("Invalid choice");  
    }  
}
```

```
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 3  
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 0  
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: 5  
Invalid choice  
Choose 0 for gogo, 1 for momo, 2 for yoyo, 3 for koko, -1 to exit: -1
```



0	1	2	3
gogo	momo	yoyo	koko

0	1	2	3
1	0	0	1

-1

המורה של המדינה :

הקוד

Each teacher and number of votes:

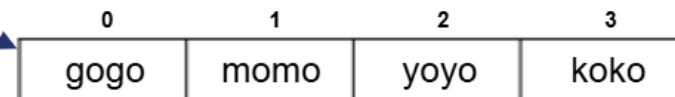
gogo --> 1 votes

momo --> 0 votes

yoyo --> 0 votes

koko --> 1 votes

```
static void PrintHowManyVotes(int[] votes, string[] names)
{
```



0	1	2	3
gogo	momo	yoyo	koko



0	1	2	3
1	0	0	1

```
    for (int i = 0; i < names.Length; i++)
    {
        Console.WriteLine(names[i] + " --> " + votes[i] + " votes");
    }
}
```


המורה של המדינה :

הקוד

The teachers are : gogo koko

0	1	2	3
gogo	momo	yoyo	koko

```
static void PrintChosenTeachers (int[] votes, string[] names)
{
```

```
    int max = Max(votes);
```

```
    Console.WriteLine ("The chosen teachers are : ");
```

```
    for (int i = 0; i < votes.Length; i++)
```

```
    {
```

```
        if (votes[i] == max)
```

```
            Console.Write (names[i] + " ");
```

```
    }
```

```
}
```

0	1	2	3
1	0	0	1

```
static int Max(int[] arr)
```

```
{
```

```
    int max = arr[0];
```

```
    for (int i = 0; i < votes.Length; i++)
```

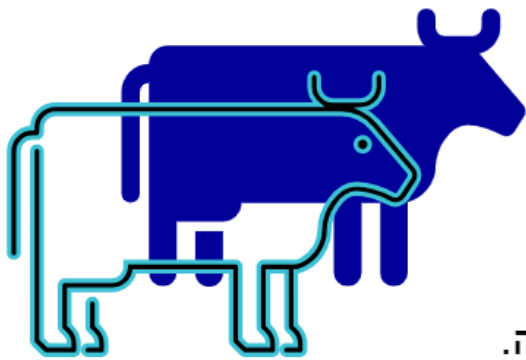
```
        max = Math.max = (max, votes[i]);
```

```
    return max;
```

```
}
```

נחלק את המשימה ל:

- מציאת הניקוד המקסימלי
- הדפסת שם/שמות המורים עם ניקוד זה.



מערך צוברים

ברפת N פרות ולכל פרה מספר סידורי בין 1 ל-N.

כל יום נרשמים נתוני החליבה, ובסוף החודש בודקים את התפוקה החודשית של כל פרה. פרה שלא הגיעה בסוף החודש למחצית התפוקה הממוצעת החודשית של כל הפרות ברפת, נבדקת על ידי וטרינר.

דרוש אלגוריתם אשר מקבל כקלט :

✓ מספר הפרות.

✓ רשימה של נתוני החליבה החודשיים (30 ימים).

פלט: מספרי הפרות שיש לשלוח לבדיקה וטרינרית.



מעריך צוברים

- על מנת לבדוק את נתוני החליבה החודשיים של כל פרה, נגדיר מעריך שגודלו כמספר הפרות N , כאשר כל תא הוא צובר נתוני החליבה עבור פרה אחת. נאתחל כל צובר (תא) ל-0.
- בכל יום בחודש, במשך 30 ימים, נקלוט את נתוני החליבה היומיים לכל אחת מהפרות, ונעדכן עבור כל אחת את הצובר "שלה" (את התא במעריך המשויך לה).

הצובר של פרה 1	הצובר של פרה 2	הצובר של פרה 3	הצובר של פרה 4
0	1	2	3
7.7	7.9	11.9	5.8

נדגים על 4 פרות במהלך יומיים. בכל יום, מעדכנים את הצוברים של כל הפרות בתפוקת החלב היומית.

מערך צוברים

- לאחר שיהיה לנו מערך הצוברים, נחשב מה ממוצע תפוקת החלב החודשי לפרה.

0	1	2	3
7.7	7.9	11.9	5.8

- נעבור על הנתונים, ונדפיס את מספרי הפרות שתפוקת החלב החודשית שלהן נמוך מממוצע תפוקת החלב של כל הפרות ברפת.
- מספרי הפרות מתחילים ב-1, והאינדקסים במערך מתחיל ב-0.
 - ✓ ניתן לבנות מערך בגודל N ואז הצובר השייך לפרה i הוא התא באינדקס $i-1$.
 - ✓ ניתן לבנות מערך בגודל $N+1$, להתעלם מתא 0, וכך האינדקס של הצובר יהיה גם מספר הפרה.

מערך צוברים – חלוקה לתת משימות

- **קלט:** מספר הפרות.

- בניית מערך צוברים, קלט של נתוני החליבה החודשיים.

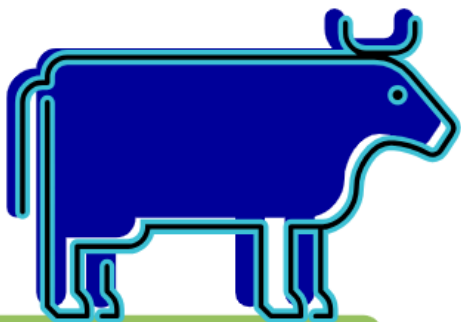
- חישוב ממוצע נתוני החליבה החודשי לפרות ברפת.

- הדפסת מספרי הפרות שתפוקת החלב החודשית שלהן קטנה מהממוצע.

פעולה המקבלת מספר פרות,
בונה מערך צוברים,
קולטת את נתוני החליבה החודשיים
ומחזירה את המערך.

פעולה המקבלת מערך
ומחזירה את הממוצע של האיברים.

- **אפשרות אחת** – ביצוע המשימה בפעולה הראשית.
- **אפשרות שניה** – פעולת void המקבלת את המערך והממוצע, ומדפיסה.
- **אפשרות שלישית** – פעולה המקבלת את המערך והממוצע ומחזירה מערך רק של מספרי הפרות שתנובת החלב שלהן קטנה מהממוצע.



מערך צוברים – חישוב הממוצע

```
// הפעולה מקבלת מערך מטיפוס ממשי  
// הפעולה מחזירה את הממוצע המערך  
  
static double ArrayAvg(double[] arr)  
{  
    double sum = 0;  
    for (int i = 0; i < arr.Length; i++)  
    {  
        sum += arr[i];  
    }  
    return sum/arr.Length;  
}
```

חישוב ממוצע של מערך אינו רלבנטי
רק לממוצע תפוקת חלב
לכן - נכתוב פעולה כללית.

מערך צוברים – הדפסת המערך

// הפעולה מקבלת מערך מטיפוס שלם
// הפעולה מדפיסה את המערך

```
static void printArray(int[] arr)
{
    Console.Write("[");
    for (int i = 0; i < arr.Length; i++)
    {
        Console.Write(arr[i] + ", ");
    }
    Console.WriteLine(arr[arr.Length - 1] + "]");
}
```

// הפעולה מקבלת מערך מטיפוס ממשי
// הפעולה מדפיסה את המערך

```
static void printArray(double[] arr)
{
    Console.Write("[");
    for (int i = 0; i < arr.Length; i++)
    {
        Console.Write(arr[i] + ", ");
    }
    Console.WriteLine(arr[arr.Length - 1] + "]");
}
```

לשם בדיקה, נכתוב פעולות המדפיסות מערכים.
שם הפעולות זהה, אך הפרמטר מטיפוס שונה
II – העמסת פעולות

מערך צוברים – צבירה במערך

גרסה 1 – מערך בגודל N

// הפעולה מקבלת מספר פרות
// הפעולה קולטת נתוני חליבה חודשיים של כל הפרות
// ומחזירה מערך ובו נתוני החליבה

אם הפעולה מייצרת מערך – היא תחזיר אותו.

```
static double[] BuildMilkArray(int numCows)
{
    double[] arr = new double[numCows];
    for (int i = 0; i < arr.Length; i++)
        arr[i] = 0;

    for (int day = 1; day <= 30; day++)
    {
        for (int cow = 1; cow <= numCows; cow++)
        {
            Console.WriteLine("day #" + day + " cow #" + cow + ": ");
            arr[cow - 1] += double.Parse(Console.ReadLine());
        }
    }
    return arr;
}
```

בניית מערך צוברים בגודל מספר הפרות.
לכל פרה מוקצה תא במערך.

יש לאפס את מערך הצוברים.
למרות שבעת יצירת מערך מספרים
ברירת המחדל היא שהמערך מאופס.

לולאה עבור כל יום בחודש.

נקלוט עבור כל פרה את נתוני החליבה היומיים,
ונוסיף אותם לצובר המשוייך לה.

מספרי הפרות מתחיל ב-1,
אך המערך מתחיל במצוין 0
לקן

המצוין של הצובר המשוייך לפרה X הוא X-1.

מערך צוברים – צבירה במערך

גרסה 2 – מערך בגודל N+1

// הפעולה מקבלת מספר פרות
// הפעולה קולטת נתוני חליבה חודשיים של כל הפרות
// ומחזירה מערך ובו נתוני החליבה
// תא 0 אינו חלק מנתוני המערך

```
static double[] BuildMilkArray2(int numCows)
{
    double[] arr = new double[numCows + 1];
    for (int i = 0; i < arr.Length; i++)
        arr[i] = 0;

    for (int day = 1; day <= 30; day++)
    {
        for (int cow = 1; cow <= numCows; cow++)
        {
            Console.Write("day #" + day + " cow #" + cow + ": ");
            arr[cow] += double.Parse(Console.ReadLine());
        }
    }
    return arr;
}
```

בגרסה זו, נבנה מערך גדול ב-1 ממספר הפרות, ונתעלם מתא 0.

איפוס את מערך הצוברים.

האינדקס שווה למספר הפרה.

בגרסה זו,
תא 0 אינו חלק ממערך הנתונים !

בניית מערך בגודל לא ידוע

// הפעולה מקבלת מערך מטיפוס ממשי וערך //

// הפעולה מחזירה מערך ובו מספרי הפרות שתפוקת החלב שלהן נמוכה מהערך //

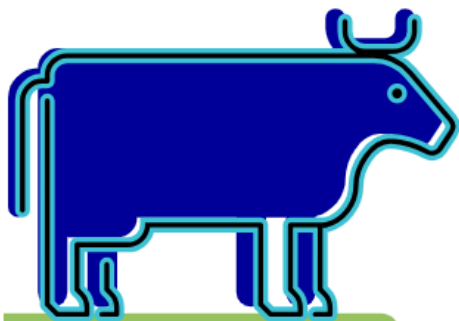
- גודל המערך החדש אינו ידוע!
איננו יודעים כמה פרות לא עמדו בתפוקה הממוצעת
- **לא ניתן לבנות מערך ללא גודל!**

✓ **אפשרות אחת** – נמנה כמה תאים עונים על התנאי ונבנה מערך בגודל מתאים.

- נעבור על המערך, נספור כמה פרות עומדות בתנאי.
- נבנה מערך בגודל המתאים.
- נעבור שוב על מערך הצוברים ונעתיק את מספרי הפרות המתאימות למערך החדש.

✓ **אפשרות שניה** – נבנה מערך בגודל מספיק.

- נבנה מערך שיהיה מספיק גדול - במקרה שלנו **N**.
- נעבור על מערך הצוברים ואם הפרה עומדת בתנאי, נעתיק את מספרה למערך.
- נעבור שוב על מערך הצוברים ונעתיק את מספרי הפרות המתאימות למערך החדש.



בניית מערך בגודל לא ידוע

// הפעולה מקבלת מערך מטיפוס ממשי וערך //

// הפעולה מחזירה מערך ובו מספרי הפרות שתפוקת החלב שלהן נמוכה מהערך //

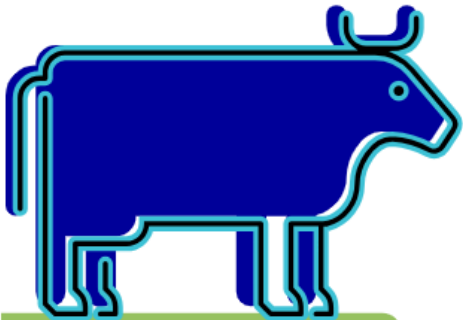
- נבחר לכתוב את הפעולה בדרך השנייה

✓ נבנה מערך שיהיה מספיק גדול – במקרה שלנו N .

✓ נעבור על מערך הצוברים ואם הפרה עומדת בתנאי, נעתיק את מספרה למערך.

✓ לאחר מכן, כאשר ידוע מספר הפרות העומדות בתנאי, נעתיק למערך בגודל הנכון את הנתונים.

משתנה המצביע על המקום הפנוי הבא
(וגם על כמות האיברים במערך החדש).



index = 3

0	1	2	3	4	5	6	7
8	2	3	7	5	5	1	9

avg = 5

0	1	2	3	4	5	6	7
1	2	6	0	0	0	0	0

מערך צוברים :

בניית מערך בגודל לא ידוע

```
// הפעולה מקבלת מערך מטיפוס ממשי וערך  
// הפעולה מחזירה מערך ובו מספרי הפרות שתפוקת החלב שלהן נמוכה מהערך  
static int[] BelowCriteria(double[] arr, double criteria)
```

```
{  
    int[] temp = new int[arr.Length];  
    int index = 0;  
    for (int i = 0; i < arr.Length; i++)  
    {  
        if (arr[i] < criteria)  
        {  
            temp[index] = i + 1;  
            index++;  
        }  
    }  
    int[] below = new int[index];  
    for (int i = 0; i < index; i++)  
    {  
        below[i] = temp[i];  
    }  
    return below;  
}
```

המיקום במערך החדש
אינו המיקום המקורי

עדכון האינדקס, כך שיצביע על
המקום הפנוי הבא

איננו יודעים מה גודל מערך התוצאה,
אך הוא לא יהיה גדול ממערך הקלט!

מספר הפרה הוא האינדקס פחות 1,
כי המערך מתחיל ב-0 ומיספור הפרות מתחיל מ-1

המשתנה הוא גם המיקום הפנוי הבא,
אך גם מספר האיברים בפועל.

העתקה למערך בגודל הנכון, והחזרתו.

שימו לב!

האינדקס במערך הנתון והאינדקס במערך החדש, שונים!

מערך צוברים :

בניית מערך בגודל לא ידוע עם פעולת עזר

// הפעולה מקבלת מערך מטיפוס ממשי וערך
// הפעולה מחזירה מערך ובו מספרי הפרות שתפוקת החלב שלהן נמוכה מהערך
`static int[] BelowCriteria(double[] arr, double criteria)`

```
{  
    int[] temp = new int[arr.Length];  
    int index = 0;  
    for (int i = 0; i < arr.Length; i++)  
    {  
        if (arr[i] < criteria)  
        {  
            temp[index] = i + 1;  
            index++;  
        }  
    }  
    return ResizeArr(temp, index);  
}
```

המיקום במערך החדש
אינו המיקום המקורי

עדכון האינדקס, כך שיצביע על
המקום הפנוי הבא

איננו יודעים מה גודל מערך התוצאה,
אך הוא לא יהיה גדול ממערך הקלט !

מספר הפרה הוא האינדקס פחות 1,
כי המערך מתחיל ב-0 ומיספור הפרות מתחיל מ-1

המשתנה index הוא מספר האיברים בפועל.

הפעולה **מייצרת** מערך חדש
ולכן - היא **מחזירה** אותו

```
static int[] ResizeArr(double[] arr, int size)  
{  
    int[] temp = new int[size];  
    for (int i = 0; i < temp.Length; i++)  
        temp[i] = arr[i];  
    return temp;  
}
```

מערך צוברים

```
Num Cows: 8
day #1 cow #1: 4.9
day #1 cow #2: 6.7
day #1 cow #3: 5.9
day #1 cow #4: 6
day #1 cow #5: 4.4
day #1 cow #6: 6.8
day #1 cow #7: 4.9
day #1 cow #8: 5.8
day #2 cow #1: 4.7
day #2 cow #2: 3.7
day #2 cow #3: 2.9
day #2 cow #4: 9.7
day #2 cow #5: 7
day #2 cow #6: 4.2
day #2 cow #7: 8.5
day #2 cow #8: 9.4
day #3 cow #1: 2.9
day #3 cow #2: 4.1
day #3 cow #3: 4.5
day #3 cow #4: 7.4
day #3 cow #5: 4.1
day #3 cow #6: 6.8
day #3 cow #7: 4.6
day #3 cow #8: 7.3
[12.500000000000002, 14.5, 13.3, 23.1, 15.5, 17.8, 18, 22.5]
avg = 17.15
[1, 2, 3, 5]
```

```
static void Main(string[] args)
{
    Console.Write("Num Cows: ");
    int numCows = int.Parse(Console.ReadLine());

    double[] milkArr = BuildMilkArray(numCows);
    printArray(milkArr);

    double avg = ArrayAvg(milkArr);
    Console.WriteLine("avg = " + avg);

    int[] cows = BelowCriteria(milkArr, avg);
    printArray(cows);
}
```

פעולה הבונה
מערך צוברים.

פעולה המחשבת
את הממוצע.

פעולה המחזירה
את הפרות שהן
מתחת לממוצע.

שאלת סיכום 1 : מספר ארוך מאד

משתנה מסוג int יכול להכיל מספרים שבין -2147483648 ל-2,147,483,647. לעיתים, יש צורך במספרים ארוכים מאד. אחת הדרכים לייצג אותם היא באמצעות מערך ספרות. הספרה הראשונה היא בתא הראשון במערך, והיא אינה 0 ושאר הספרות יהיו בשאר תאי המערך, לפי הסדר. לדוגמה:

המספר 45677021792213278809 ייוצג ע"י המערך :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
4	5	6	7	7	0	2	1	7	9	2	2	1	3	2	7	8	8	0	9

- כתבו פעולה `StringToLongNumber` המקבלת מחרוזת ספרות ומחזירה מערך ספרות המייצג את המחרוזת הארוכה מאד.
- כתבו פעולה `CompareTo` המקבלת שני מספרים גדולים (מערכי ספרות), ומחזירה 1 אם המספר הראשון גדול מהשני, -1 אם השני גדול מהראשון, ו-0, אם הם שווים.
- כתבו פעולה `Add` המקבלת שני מספרים גדולים (מערכי ספרות), ומחזירה את מערך הספרות שהוא תוצאת החיבור של שני המספרים.

שאלת סיכום 1 : מספר ארוך מאד

א. כתבו פעולה `StringToLongNumber` המקבלת מחרוזת ספרות ומחזירה מערך ספרות המייצג את המחרוזת הארוכה מאד.

Dec	Hex	Char	Dec	Hex	Char
32	20	Space	64	40	@
33	21	!	65	41	A
34	22	"	66	42	B
35	23	#	67	43	C
36	24	\$	68	44	D
37	25	%	69	45	E
38	26	&	70	46	F
39	27	'	71	47	G
40	28	(	72	48	H
41	29	)	73	49	I
42	2A	*	74	4A	J
43	2B	+	75	4B	K
44	2C	,	76	4C	L
45	2D	-	77	4D	M
46	2E	.	78	4E	N
47	2F	/	79	4F	O
48	30	0	80	50	P
49	31	1	81	51	Q
50	32	2	82	52	R
51	33	3	83	53	S
52	34	4	84	54	T
53	35	5	85	55	U
54	36	6	86	56	V
55	37	7	87	57	W
56	38	8	88	58	X
57	39	9	89	59	Y
58	3A	:	90	5A	Z
59	3B	;	91	5B	[
60	3C	<	92	5C	\
61	3D	=	93	5D	]
62	3E	>	94	5E	^
63	3F	?	95	5F	_

הפעולה מקבלת מחרוזת,
מכיון ש-int מוגבל למקסימום 10 ספרות.

```
static int[] StringToLongNumber(string s)
{
    int[] num = new int[s.Length];
    for (int i = 0; i < s.Length; i++)
    {
        num[i] = s[i] - '0';
    }
    return num;
}
```

המרה של התו שבמחרוזת לספרה

שאלת סיכום 1 : מספר ארוך מאד

ב. כתבו פעולה `CompareTo` המקבלת שני מספרים גדולים (מערכי ספרות), ומחזירה :
1 אם המספר הראשון גדול מהשני,
-1 אם השני גדול מהראשון,
0 אם הם שווים.

```
static int CompareTo(int[] num1, int[] num2)
{
    if (num1.Length > num2.Length)
        return 1;
    if (num1.Length < num2.Length)
        return -1;

    int i = 0;
    // לולאת for ריקה, מקדמת את i כל עוד הערכים במערך שווים
    for (i < num1.Length && num1[i] == num2[i] ; i++) ;

    if (i == num1.Length)
        return 0;
    if (num1[i] > num2[i])
        return 1;
    return -1;
}
```

אם מספר ארוך מהאחר הוא הגדול יותר.

נעבור בלולאה על המספרים במקביל, ונחפש ספרה שונה.

אם קיימת ספרה שונה, המספר הגדול יותר הוא זה עם הספרה הגדולה, אחרת, המספרים שווים.

0	1	2	3	4	5	6
4	5	6	7	7	0	2

0	1	2	3	4
4	5	6	7	7

0	1	2	3	4	5	6
4	5	6	7	7	0	2

0	1	2	3	4	5	6
4	5	6	7	3	0	2

שאלת סיכום 1 : מספר ארוך מאד

```
static void Main(string[] args)
{
    int[] a1 = StringToLongNumber("9876565432546264");
    printArray(a1);
    int[] a2 = StringToLongNumber("9876565432");
    printArray(a2);
    Console.WriteLine("CompareTo = " + CompareTo(a1, a2));

    a1 = StringToLongNumber("98765654325");
    printArray(a1);
    a2 = StringToLongNumber("9876565432546264");
    printArray(a2);
    Console.WriteLine("CompareTo = " + CompareTo(a1, a2));

    a1 = StringToLongNumber("9876565432546264");
    printArray(a1);
    a2 = StringToLongNumber("9876565432548264");
    printArray(a2);
    Console.WriteLine("CompareTo = " + CompareTo(a1, a2));
}
```

```
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2, 5, 4, 6, 2, 6, 4]
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2]
CompareTo = 1
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2, 5]
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2, 5, 4, 6, 2, 6, 4]
CompareTo = -1
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2, 5, 4, 6, 2, 6, 4]
[9, 8, 7, 6, 5, 6, 5, 4, 3, 2, 5, 4, 8, 2, 6, 4]
CompareTo = -1
```

שאלת סיכום 1 : מספר ארוך מאד

ג. כתבו פעולה Add המקבלת שני מספרים גדולים (מערכי ספרות), ומחזירה את מערך הספרות שהוא תוצאת החיבור של שני המספרים.

$$\begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 9 & 9 & 9 & 9 & 9 & 0 & 3 & 5 \\ + & & & & & & & \\ & & & & & & 4 & 5 & 6 & 7 \\ \hline 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 0 & 0 & 0 & 0 & 3 & 6 & 0 & 2 \end{array}$$

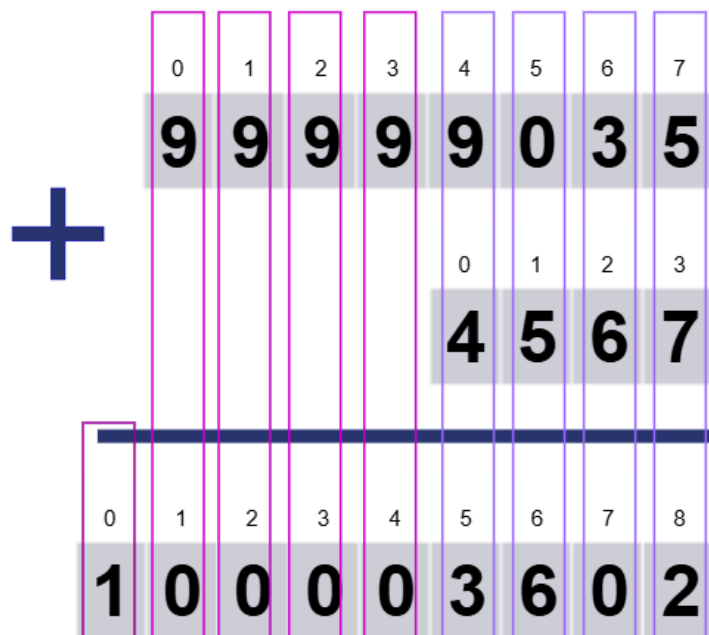
התוצאה עשויה להיות ארוכה, לכל היותר ב-1 מהמערך הארוך יותר.

$$\begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 4 & 5 & 6 & 7 & 7 & 0 & 5 & 1 \\ + & & & & & & & \\ & & & & & & 4 & 5 & 6 & 7 \\ \hline 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 4 & 5 & 6 & 8 & 1 & 6 & 1 & 8 \end{array}$$

התוצאה תשמר במערך ספרות. מה גודל המערך ?

שאלת סיכום 1 : מספר ארוך מאד

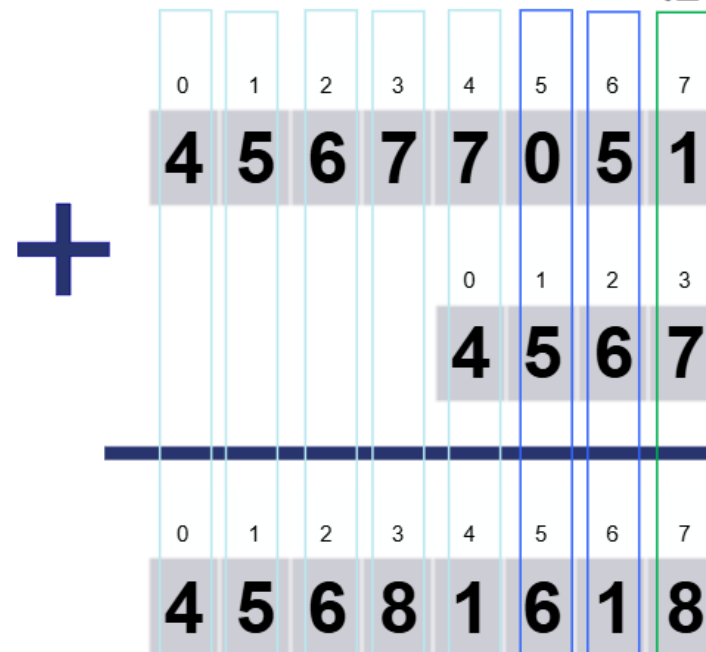
ג. כתבו פעולה Add המקבלת שני מספרים גדולים (מערכי ספרות), ומחזירה את מערך הספרות שהוא תוצאת החיבור של שני המספרים.



אם יש צורך,
נטפל באיבר הנוסף.

המשך המעבר על
המערך הארוך.

הלולאה - באורך
המערך הקטן.



אם התוצאה גדולה מ-9, תכתב
רק ספרת האחדות, ונעביר 1
(ספרת העשרות) לתאים הבאים.

נחבר ספרות מסוף
המערכים לתחילתם.

שאלת סיכום 1 : מספר ארוך מאד

```
static int[] Add(int[] num1, int[] num2)
{
    int[] temp = new int[Math.Max(num1.Length, num2.Length) + 1];
```

```
    int i1 = num1.Length - 1;
    int i2 = num2.Length - 1;
    int i3 = temp.Length - 1;
```

החיבור יתבצע
מהסוף להתחלה.

המערך החדש יהיה
בגודל המערך הגדול + 1

נשא - הערך שיש
להעביר במקרה
שסכום הספרות
גדול מ-9

```
    int sum, carry = 0;
```

```
    while (i1 >= 0 && i2 >= 0)
    {
```

לולאה באורך המערך הקצר.

```
        sum = num1[i1] + num2[i2] + carry;
```

```
        temp[i3] = sum % 10;
```

```
        carry = sum / 10;
```

מחברים את שני התאים עם הנשא carry,
ספרת האחדות היא הערך של התוצאה,
ספרת העשרות היא ה-carry הבא.

```
        i1--;
```

```
        i2--;
```

```
        i3--;
```

```
    }
```

שאלת סיכום 1 : מספר ארוך מאד

```
while (i1 >= 0)
{
    sum = num1[i1] + carry;
    temp[i3] = sum % 10;
    carry = sum / 10;
    i1--;
    i3--;
}
while (i2 >= 0)
{
    sum = num2[i2] + carry;
    temp[i3] = sum % 10;
    carry = sum / 10;
    i2--;
    i3--;
}
```

או שנשתמש בפעולה במקום לחזור על הקוד

```
carry = AddRest (num1, i1, sum, i3, carry);
carry = AddRest (num2, i2, sum, i3, carry);
```

אם אחד המערכים ארוך יותר,
אחת הלולאות תטפל בהמשך הספרות.
אין טעם לבדוק מי מהמערכים ארוך יותר,
כיוון שתנאי הלולאה בודק זאת.

```
static int AddRest (int[] num, int i,
                    int[] sum, int j, int carry)
{
    while (i >= 0)
    {
        sum = num[i] + carry;
        temp[j] = sum % 10;
        carry = sum / 10;
        i--;
        j--;
    }
    return carry;
}
```

שאלת סיכום 2 – שאלת בגרות

חברת האינטרנט "אינטרגיל" עורכת סקר שביעות רצון בין לקוחותיה. הסקר נערך פעם בחודש. **לקוח המשתתף בסקר נותן לחברה ציון שביעות רצון שהוא מספר שלם בין 1 ל-10 (כולל).**

אם במהלך שנה אחת יש **חודשיים רצופים** שבכל אחד מהם הממוצע של ציוני שביעות הרצון של הלקוחות **גדול מ-8**, החברה מזמינה את עובדיה ל"יום כיף".

א. כתוב פעולה שתקלוט בעבור חודש מסוים את ציוני שביעות הרצון של הלקוחות שהשתתפו בסקר. הפעולה תחזיר את הציון הממוצע של שביעות הרצון של הלקוחות באותו חודש. קליטת הציונים תסתיים כאשר ייקלט 1- בעבור ציון שביעות הרצון. אם בחודש זה השתתפו בסקר פחות מ-100 לקוחות, הציון שיוחזר בעבור ממוצע ציוני שביעות הרצון יהיה 0.

ב. כתוב פעולה שתקבל מערך בגודל 12, המכיל בעבור כל חודש את הציון הממוצע של שביעות הרצון של הלקוחות באותו חודש. אם העובדים בחברה זכאים ל"יום כיף", הפעולה תחזיר true, אחרת – היא תחזיר false.

ג. כתוב תכנית שתקלוט בעבור כל אחד מהחודשים בשנה מסוימת את התוצאות של סקר שביעות הרצון של הלקוחות. התכנית תבדוק אם העובדים זכאים ל"יום כיף" או לא, ותדפיס הודעה מתאימה.

שאלת סיכום 2 – שאלת בגרות חלק א'

חברת האינטרנט "אינטרגיל" עורכת סקר שביעות רצון בין לקוחותיה. הסקר נערך פעם בחודש. **לקוח המשתתף בסקר נותן לחברה ציון שביעות רצון שהוא מספר שלם בין 1 ל-10 (כולל).** אם במהלך שנה אחת יש **חודשיים רצופים** שבכל אחד מהם הממוצע של ציוני שביעות הרצון של הלקוחות **גדול מ-8**, החברה מזמינה את עובדיה ל"יום כיף".

- א. כתוב פעולה שתקלוט בעבור חודש מסוים את **ציוני שביעות הרצון** של הלקוחות שהשתתפו בסקר. הפעולה תחזיר את **הציון הממוצע** של שביעות הרצון של הלקוחות באותו חודש. **קליטת הציונים תסתיים כאשר ייקלט 1-** בעבור ציון שביעות הרצון. אם בחודש זה השתתפו בסקר **פחות מ-100 לקוחות**, הציון שיוחזר בעבור ממוצע ציוני שביעות הרצון **יהיה 0**.

שאלת סיכום 2 – שאלת בגרות חלק א'

```
static double GetMonthlyAverageScore()
{
    int sum = 0;
    int count = 0;

    Console.WriteLine("Enter the monthly average satisfaction score (1-10, or -1 to exit): ");
    int score = int.Parse(Console.ReadLine());

    while (score != -1)
    {
        sum += score;
        count++;
        Console.WriteLine("Enter the monthly average satisfaction score (1-10, or -1 to exit): ");
        score = int.Parse(Console.ReadLine());
    }

    if (count < 100)
        return 0; // Less than 100 participants, return 0 as specified

    return (double)sum / count;
}
```

אתחול מונה וצובר עבור ממוצע

קלט ראשון לפני הלולאה

לולאת זקיף

עדכון מונה וצובר

קלט נוסף בסוף הלולאה

המרה לצורך חישוב
הממוצע כמספר ממשי

שאלת סיכום 2 – שאלת בגרות חלק ב'

חברת האינטרנט "אינטרגיל" עורכת סקר שביעות רצון בין לקוחותיה. הסקר נערך פעם בחודש. לקוח המשתתף בסקר נותן לחברה ציון שביעות רצון שהוא מספר שלם בין 1 ל-10 (כולל). אם במהלך שנה אחת יש חודשיים רצופים שבכל אחד מהם הממוצע של ציוני שביעות הרצון של הלקוחות גדול מ-8, החברה מזמינה את עובדיה ל"יום כיף".

ב. כתוב פעולה שתקבל מערך בגודל 12, המכיל בעבור כל חודש את הציון הממוצע של שביעות הרצון של הלקוחות באותו חודש. אם העובדים בחברה זכאים ל"יום כיף", הפעולה תחזיר true, אחרת – היא תחזיר false.

```
static bool CheckFunDayEligibility (double[] monthlyAverageScores)
{
    for (int i = 0; i < monthlyAverageScores.Length - 1 ; i++)
    {
        if (monthlyAverageScores[i] > 8 && monthlyAverageScores[i + 1] > 8)
        {
            return true;
        }
    }
    return false;
}
```

כאשר פונים לאינדקס אחר מאשר i, חייבים לוודא שלא תהיה חריגה מהמערך

שאלת סיכום 2 – שאלת בגרות חלק ג'

חברת האינטרנט "אינטרגיל" עורכת סקר שביעות רצון בין לקוחותיה. הסקר נערך פעם בחודש. **לקוח המשתתף בסקר נותן לחברה ציון שביעות רצון שהוא מספר שלם בין 1 ל-10 (כולל).** אם במהלך שנה אחת יש **חודשיים רצופים** שבכל אחד מהם הממוצע של ציוני שביעות הרצון של הלקוחות **גדול מ-8**, החברה מזמינה את עובדיה ל"יום כיף".

ג. כתוב תכנית שתקלוט בעבור כל אחד מהחודשים בשנה מסוימת את התוצאות של סקר שביעות הרצון של הלקוחות. **התכנית תבדוק אם העובדים זכאים ל"יום כיף" או לא**, ותדפיס הודעה מתאימה. **יש להשתמש בפעולות מסעיפים א-ב.**
הערה: אין צורך לבדוק את תקינות הקלט.

שאלת סיכום 2 – שאלת בגרות חלק ג'

```
static void Main(string[] args)
{
    double[] monthlyAverageScores = new double[12];

    // Input monthly average scores
    for (int i = 0; i < 12; i++)
    {
        monthlyAverageScores[i] = GetMonthlyAverageScore();
    }

    // Check if employees are eligible for "Fun Day"
    bool isFunDayEligible = CheckFunDayEligibility(monthlyAverageScores);

    string s = "";
    if (!isFunDayEligible) s = " not";

    Console.WriteLine("Employees are" + s + " eligible for Fun Day.");
}
```

הפעולה מסעיף א

הפעולה מסעיף ב

הדפסת הודעה מתאימה

מיונים וחיפושים

מיונים

0	1	2	3	4
3	6	12	65	76

0	1	2	3
89	77	56	50

0	1	2	3	4	5	6	7
2.4	4.5	6.8	6.9	7.1	12.3	14.6	18.6

Find 548

819	764	871	378	182	650		405		201	851	555	96	606		600	277	558	959		
366	730	795	449	764	768	90	608	66	655	76	627	241	237	670	975	294		38	50	
5	979	100	21	926	334	52	27	16	267	383	83	930	136	16	280	562	893	56	161	168
60	936	961	3	900	299	74	10	43	592	684	288	432		515	553	220	325	869	661	
907	91	695	58	557	77	587	609	147	610	648		227	342	941	819	459	923	965	670	
69	648	60	644	183	507	815	240	707	161				114	902	885	292	534	184	202	2
802	993	833	324	921	722	955	972	330	85		249	631	281	574	191	636	163	768	799	1
27	572	290	538	772	355	859	826	893	31	998	594	166	247	224	3		279		733	929
36	491	234		180	232	893	608	324	398	487	966	579	64	52	335	752	888	986	857	853
539	439	959	957	948	449	889	695	428	375	994	781	742	48	534	959	311	197	555	725	4
163	694	417	580	117	784	340	974	400	322	744	917	821	225	360	40	687	921	761	877	
777	608	212	343	591	946	467	958	728	259	895	912	512	781		149	187	618	994	64	
5	442	345	211	307	554		273	201	654	835	939	950	182	644	275	1	923		332	261
611	926	28	587	965	140	453	848	819	816	376	1	682	487	249	259	887	983		85	849
4	750	896	842	494	920	241	838	754	0	350	99	11	810	352	921	447	232	95	304	509
	123	424	417	231	990	564	291	651	682	614	76	746	781	544		271	862	500	737	4
562	630	585	559	493	812	323	332	594	583	385	325	930		877	339	795	586	63	410	
925	207	368	177	135		897	503	336	543	91	650	464	951	300	268	62	453	573	257	10
912	69	43	139	2	736	743	745	607	311	404	606	385	438	476	324	942	548	608	895	2
83	12	979	257	429	259	776	618	9	701	821	236	172	91	449	811	410	437	599	615	
194	800	84	790	958	881	423	380	545	480	248	933	819	381		957	17	471	341	479	7
926	394	334	272	728	356	783	694	294	271	881	971	47	185	112	373	825	11	381	694	
554	59	180	759	57	123	799	294	284	90	34	37	990	841		566	719	412	312	234	273
231	666	208	944	219	19	245	951	784	54	391	273	447	907	647	479	777	210	832	246	0
389	710	146	624	873	808	137		205	327	271	122	432		909	480	704	208	248	683	
708	742	487	508	582	841	272	264	11	400	02	602		25	227	585	450	24	842	224	42

Find 548



819	764	871	378	182	650		405		201	851	555	96	606		600	277	558	959	
366	730	795	449	764	768	90	608	66	655	76	627	241	237	670	975	294		38	500
5	979	100	21	926	334	52	27	5	267	383	83	930	136	16	280	562	893	56	161
50	936	961	3	900	299	74	0	4	3	592	684	288	432		515	553	220	325	869
907	9	695	58	557	77	587	609	147	610	648		227	342	941	819	459	923	965	670
59	648	60	644	183	507	815	240	707	161					114	902	885	292	534	184
802	993	833	324	921	722	955	972	330	85		249	631	281	574	191	636	163	768	799
27	572	290	538	772	355	859	826	893	31	998	594	166	247	224	3		279		733
86	491	234		180	232	893	608	324	398	487	966	579	64	52	335	752	888	986	857
539	439	959	957	948	449	889	695	428	375	994	781	742	48	534	959	311	197	555	725
163	694	417	580	117	784	340	974	400	322	744	917	821	225	360	40	687	921	761	877
777	608	212	343	591	946	467	958	728	259	895	912	512	781		149	187	618	994	64
5	442	345	211	307	554		273	201	654	835	939	950	182	644	275	1	923		332
611	926	28	587	965	140	453	848	819	816	376	1	682	487	249	259	887	983		85
4	750	896	842	494	920	241	838	754	0	350	99	11	810	352	921	447	232	95	304
	123	424	417	231	990	564	291	651	682	614	76	746	781	544		271	862	500	737
562	630	585	559	493	812	323	332	594	583	385	325	930		877	339	795	586	63	410
925	207	368	177	135		897	503	336	543	91	650	464	951	300	268	62	453	573	257
	912	69	43	139	2	736	743	745	607	311	404	606	385	438	476	324	94	548	008
83	12	979	257	429	259	776	618	9	701	821	236	172	91	449	811	410	537	509	615
	194	800	84	790	958	881	423	380	545	480	248	933	819	381		957	17	471	341
	926	394	334	272	728	356	783	694	294	271	881	971	47	185	112	373	825	11	381
554	59	180	759	57	123	799	294	284	90	34	37	990	841		566	719	412	312	234
231	666	208	944	219	19	245	951	784	54	391	273	447	907	647	479	777	210	832	246
	389	710	146	624	873	808	137		205	327	271	122	432		909	480	704	208	248
708	742	487	508	582	841	272	264	11	400	92	602		25	227	585	450	24	842	224

Find 438



מסקנות

- ברשימה לא ממוינת חיפוש ערך דורש זמן רב.
- ברשימה לא ממוינת חיפוש ערך שאיננו קיים מחייב מעבר על כל הרשימה.

חיפוש ערך יהיה מהיר משמעותית ברשימה ממוינת מאשר ברשימה לא ממוינת,

וחיפוש ערך לא קיים הינו מהיר באותה המידה.





מיון : סוגים

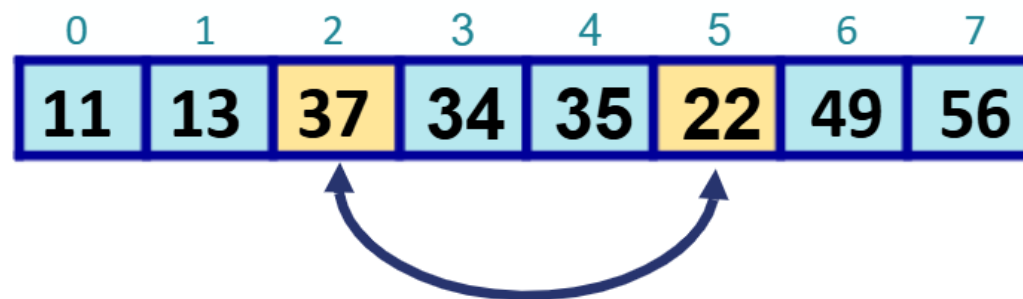
- קיימים אלגוריתמים שונים למיון נתונים.
 - האלגוריתמים שונים בזמן שלוקח להם לבצע את משימת המיון מבחינת יעילות הקוד וכן בתלות בסוג הנתונים - כמו למשל :
 - האם הנתונים אקראיים,
 - האם הנתונים ממוינים חלקית,
 - האם הנתונים ממוינים הפוך
 - ועוד ...
- 
- 

אלגוריתמים למיון : רשימה חלקית

1. **Bubble Sort (מיון בועות)** - מיון בועות הוא אלגוריתם פשוט שעובר דרך הרשימה פעמים רבות, ובכל פעם מחליף בין שני איברים אם הם לא בסדר הנכון.
2. **Selection Sort (מיון בחירה)** - מיון בחירה הוא אלגוריתם שבו הוא בודק את רשימת האיברים, בוחר את האיבר הקטן ביותר ומעביר אותו לתחילת הרשימה.
3. **Insertion Sort (מיון הכנסה)** - מיון הכנסה הוא אלגוריתם שבו הוא מקבלים את הערכים ומכניסים אותם למקומם הנכון ברשימה.
4. **Merge Sort (מיון מיזוג)** - מיון מיזוג הוא אלגוריתם רקורסיבי המחלק את הרשימה לחצאים, ממין את כל חצי בנפרד ואז ממזג את החצאים לרשימה אחת ממוינת.
5. **Quick Sort (מיון מהיר)** - מיון מהיר הוא אלגוריתם רקורסיבי שבו הוא בוחר איבר מוביל (pivot) ומסדר את הרשימה לפי האיבר המוביל, כך שכל האיברים הגדולים ממנו בצד אחד ואלה הקטנים ממנו בצד השני.

מיון

- מיון מערך מבוסס על החלפת ערכי תאים שאינם מסודרים:



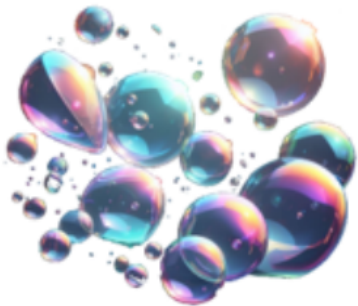
פעולת עזר בה נשתמש בהמשך

// הפעולה מקבלת מערך ושני אינדקסים
// הפעולה מחליפה בין ערכי התאים

```
static void Swap(int[] arr, int i, int j)
{
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

- אם נחליף בין תא 2 לתא 5, נקבל מערך ממוין

הפעולה מקבלת את המערך כפרמטר
לכן,
כל שינוי בפעולה משפיע על המערך המקורי.



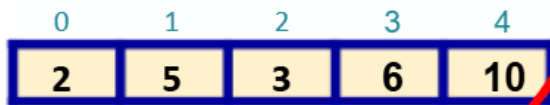
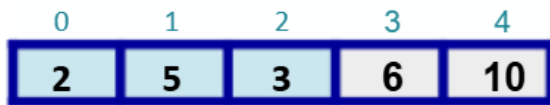
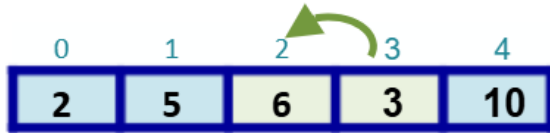
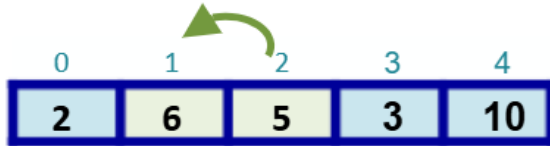
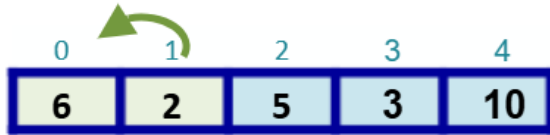
מיון בועות Bubble Sort

אלגוריתם פשוט שעובר על המערך פעמים רבות,
ובכל פעם מחליף בין שני איברים אם הם לא בסדר הנכון.

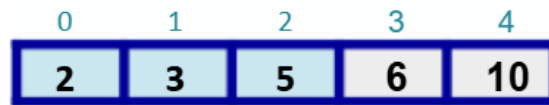
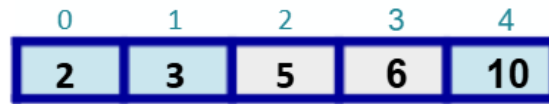
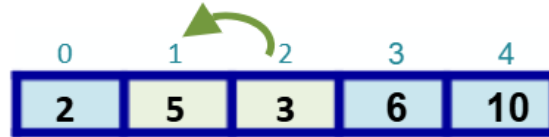
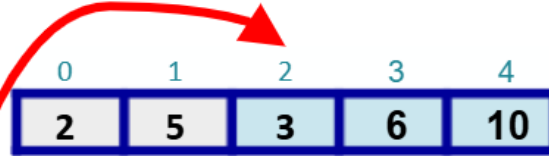
- כל עוד המערך אינו ממוין
 - ניקח את שני הערכים הראשונים במערך ונשווה ביניהם (הערכים בתא 0 ובתא 1).
 - נבדוק אם הערך בתא הראשון גדול יותר מהערך בתא השני ונבצע החלפה במידת הצורך.
 - נתקדם איבר אחד ונחזור על התהליך לעיל עד שהמערך יהיה ממוין.
- האלגוריתם נקרא כך כי בכל מעבר על המערך, **האיבר הגדול ביותר מגיע למקומו**.
במעבר הראשון הגדול ביותר מגיע לסוף המערך, במעבר השני, השני בגודלו מגיע לתא לפני אחרון וכך הלאה.
ערכי המערך "מפעפעים" כמו בועות אוויר במים – הגדולות לפני הקטנות.



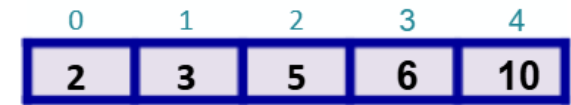
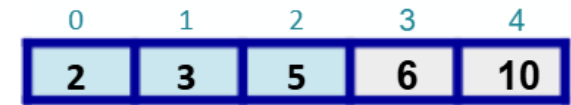
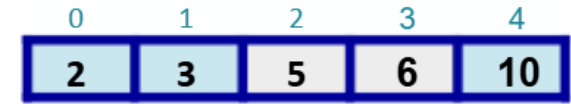
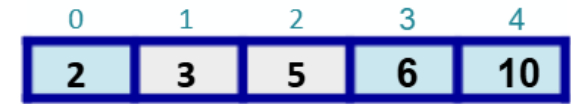
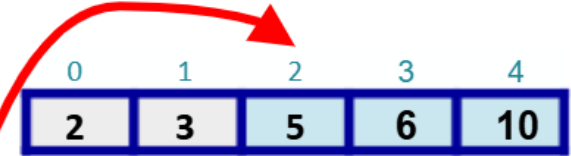
מיזן בועות Bubble Sort



סיימנו מעבר שלם על המערך,
האיבר הגדול ביותר הגיע לסוף המערך.



סיימנו מעבר שלם על המערך,
האיבר השני בגודלו הגיע למקום לפני אחרון.



בצענו מעבר נוסף והתברר שהמערך
ממוין – לא בוצעו החלפות.



מיון בועות Bubble Sort

```
static void BubbleSort(int[] arr)
{
    for (int i = 0; i < arr.Length; i++)
    {
        for (int j = 0; j < arr.Length - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
                Swap(arr, j, j + 1);
        }
    }
}
```

האיבר הגדול בכל סיבוב מגיע
למקומו ולכן ניתן לעבור עד אליו

פעולת עזר



מיון בועות Bubble Sort

שיפור בזמן הריצה

```
static void BubbleSort(int[] arr)
{
    bool sorted = false;
    for (int i = 0; i < arr.Length && !sorted; i++)
    {
        bool swapRequired = false;
        for (int j = 0; j < arr.Length - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                Swap(arr, j, j + 1);
                swapRequired = true;
            }
        }
        if (swapRequired == false)
            sorted = true;
    }
}
```

פעולת עזר

אם במהלך מעבר על המערך,
לא בצענו החלפה,
המערך ממויין!



מיון בועות Bubble Sort

בשילוב פעולת עזר

```
static void BubbleSort (int[] arr)
{
    bool sorted = false;
    for (int i = 0; i < arr.Length; && !sorted; i++)
    {
        sorted = !BubbleSort(arr, arr.Length-i);
    }
}
```

האיבר הגדול בכל סיבוב מגיע למקומו
ולכן באמצעות פעולת עזר
ניתן לעבור עד אליו.
וזאת כל עוד שבוצעה החלפה בפעולה.

פעולת עזר שעוברת על המערך פעם אחת
באורך הנדרש בלבד,
ומחזירה האם בוצעה החלפה

אם במהלך מעבר על המערך,
לא בצענו החלפה,
המערך ממויין!

```
static bool BubbleSort (int[] arr, int length)
{
    bool swap = false;
    for (int i = 0; i < length-1; i++)
    {
        if (arr[i] > arr[i + 1])
        {
            Swap(arr, i, i + 1);
            swap = true;
        }
    }
    return swap;
}
```

פעולת עזר

מיון בחירה Selection Sort

בכל שלב, נמצא את הערך הקטן הנוכחי, ונעביר אותו למקומו הנכון.
הערך שהיה במקום זה יועבר למיקום המקורי של הערך הקטן.

- עבור כל איבר במערך
 - מחפשים את האינדקס של הערך המינימלי ממקום זה ואילך
 - המינימום מוחלף עם האיבר
- בכל שלב, המערך מחולק לשני חלקים, כאשר צידו השמאלי ממוין, ובכל שלב החלק הממוין גדל באיבר אחד, והחלק הלא ממוין קטן באיבר אחד.

מיון בחירה Selection Sort

0	1	2	3	4	5	6
9	5	8	12	10	4	2

0	1	2	3	4	5	6
2	5	8	12	10	4	9

0	1	2	3	4	5	6
2	4	8	12	10	5	9

0	1	2	3	4	5	6
2	4	5	12	10	8	9

0	1	2	3	4	5	6
2	4	5	8	10	12	9

0	1	2	3	4	5	6
2	4	5	8	9	12	10

0	1	2	3	4	5	6
2	4	5	8	9	10	12

מיון בחירה Selection Sort

```
static void SelectionSort(int[] arr)
{
    int minIndex;
    for (int i = 0; i < arr.Length; i++)
    {
        minIndex = i;
        for (int j = i + 1; j < arr.Length; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        Swap(arr, i, minIndex);
    }
}
```

בכל שלב, הטווח קטן

ערך נלווה למינימום -
האינדקס של האיבר הקטן בטווח,
עבור ההחלפה בין ערכי התאים

פעולת עזר

מיון בחירה Selection Sort

פיצול לפעולות עזר

```
static void SelectionSort(int[] arr)
{
    int minIndex;
    for (int i = 0; i < arr.Length; i++)
    {
        minIndex = Imin(arr, i + 1);
        Swap(arr, i, minIndex);
    }
}
```

בכל שלב, הטווח קטן

פעולת עזר

```
// פעולת עזר המקבלת מערך ואינדקס התחלה
// ומחזירה את האינדקס של האיבר המינימלי
static int Imin (int[] arr, int start)
{
    int imin = start;
    for (int i = start+1; i < arr.Length; i++)
    {
        if (arr[i] < arr[imin])
            imin = i;
    }
    return imin;
}
```

מיונים וחיפושים

חיפושים

0	1	2	3	4
3	6	12	65	76

0	1	2	3
89	77	56	50

0	1	2	3	4	5	6	7
2.4	4.5	6.8	6.9	7.1	12.3	14.6	18.6

חיפוש ערך במערך – חיפוש סדרתי

```
static bool Search(int[] input, int number)
{
    for (int i = 0; i < input.Length; i++)
    {
        if (input[i] == number)
            return true;
    }
    return false;
}
```

- כאשר אנו מחפשים איבר במערך שאינו ממוין, עלינו לעבור (במקרה הגרוע), על כל איברי המערך, על מנת להגיע למסקנה כי האיבר לא במערך.

```
static bool SearchSorted(int[] sorted, int number)
{
    for (int i = 0; i < sorted.Length && sorted[i] <= number; i++)
    {
        if (sorted[i] == number)
            return true;
    }
    return false;
}
```

- כאשר אנו מחפשים איבר במערך ממוין, אנו יכולים להסתמך על הנתון כי המערך ממוין, ואם הגענו לאיבר הגדול מזה שאנו מחפשים, ברור כי האיבר שאנו מחפשים לא נמצא במערך.



האלגוריתם נקרא גם
חיפוש אריה במדבר

חיפוש בינארי Binary Search

כאשר המערך ממויין, ניתן להשתמש באלגוריתם לחיפוש בינארי:

1. נבדוק את **האיבר האמצעי** מבין האיברים:

1.1 אם האיבר האמצעי הוא האיבר המבוקש, הרי מצאנו את שחיפשנו ונסיים כאן.

1.2 אם לא, נבדוק מה יחס הסדר בין האיברים:

1.2.1 אם האיבר המבוקש **קטן יותר** מאיבר זה, נבחר את **החצי השמאלי** של האיברים (בו כל האיברים הקטנים מהאיבר האמצעי).

1.2.2 אם האיבר המבוקש **גדול יותר**, נבחר את **החצי הימני** של האיברים (בו כל האיברים הגדולים מהאיבר האמצעי).

2. כעת **נחזור** על כל מה שעשינו עם **תת-המערך** שבחרנו.

במקרה הגרוע ביותר, שבו באף אחד מהמקרים לא היה האיבר האמצעי האיבר אותו אנו מבקשים, נגיע לתת-מערך בעל איבר אחד, שאם הוא האיבר המבוקש, **מצאנו**, ואם לא – האיבר המבוקש **לא נמצא** במערך.



חיפוש בינארי Binary Search

האם הערך 33 נמצא במערך ?



6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

↑



חיפוש בינארי Binary Search

אם היינו מחפשים את 32,
היינו יודעים בשלב זה שהמספר איננו.



6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

↑



חיפוש בינארי Binary Search

בחיפוש **סדרתי**, אנו עלולים לעבור על כל N איברי המערך.

בחיפוש **בינארי**, הטווח הנבדק קטן בכל שלב **לחצי** הטווח הקודם, ולכן מספר החיפושים יהיה שווה, לכל היותר, למספר הפעמים בהם ניתן לחלק את N ב-2,

$$2^x = N \quad \text{כלומר הפעולה המתמטית } \log$$
$$x = \log_2 N$$



חיפוש בינארי Binary Search

// פעולה המקבלת מערך ממויין ואיבר

// הפעולה תחזיר את אינדקס האיבר, או -1, אם לא נמצא במערך

```
static int BinarySearch (int[] input, int number)
```

```
{
```

```
    int low = 0;
```

```
    int high = input.Length - 1;
```

```
    int middle;
```

```
    while (high >= low)
```

```
    {
```

```
        middle = (low + high) / 2;
```

```
        if (input[middle] == number)
```

```
            return middle;
```

```
        else if (input[middle] < number)
```

```
            low = middle + 1;
```

```
        else if (input[middle] > number)
```

```
            high = middle - 1;
```

```
    }
```

```
    return -1; // not found
```

```
}
```

הטווח הראשוני - כל המערך

כל עוד **low**, האינדקס הנמוך, קטן מ-**high**, האינדקס הגבוה, עדיין יש איברים שלא נבדקו

חישוב המיקום של האיבר האמצעי בטווח

מצאנו, נחזיר את האינדקס של האיבר

האיבר האמצעי **קטן** מהאיבר שמחפשים, **נמשיך לחפש** במערך אחרי middle

האיבר האמצעי **גדול** מהאיבר שמחפשים, **נמשיך לחפש** במערך לפני middle



ביחידה זו למדנו:

- מהו מערך
- כיצד מערך נראה בזיכרון
- גישה לאיברי המערך
- אתחול מערך
- חריגה מגבולות המערך
- השמת מערכים
- מערכים כפרמטרים לפעולות
- החזרת מערך מפעולה
- מערך מונים
- מערך צוברים

• דוגמאות אלגוריתמיות:

- ✓ סיבוב מערך
- ✓ הדפסת תווים
- ✓ מיזוג מערכים ממויינים
- ✓ מספר ארוך מאד
- ✓ שאלת בגרות

• מיונים וחיפושים

- ✓ סוגי מיונים
- ✓ מיון בועות
- ✓ מיון בחירה
- ✓ חיפוש בינארי